

[Session -1](#)

[Session -2](#)

[Session -3](#)

[Session -4](#)

[Session -5](#)

[Session -6](#)

[Session -7](#)

[Session -8](#)

[Session -9](#)

[Session -10](#)

[Session -11](#)

[Session -12](#)

[Session -13](#)

[Session -14](#)

[Session -15](#)

[Session -16](#)

[Session -17](#)

[Session -18](#)

[Session -19](#)

[Session -20](#)

[Session -21](#)

[Session -22](#)

[Session -23](#)

[Session -24](#)

[Session -25](#)

[Session -26](#)

[Session -27](#)

[Session -28](#)

[Session -29](#)

[Session -30](#)

[Session -31](#)

[Session -32](#)

[Session -33](#)

[Session -34](#)

[Session -35](#)

[Session -36](#)

[Session -37](#)

[Session -38](#)

[Session -39](#)

[Session -40](#)

[Session -41](#)

[Session -42](#)

[Session -43](#)

[Session -44](#)

[Session -45](#)

[Session -46](#)

[Session -47](#)

[Session -48](#)

[Session -49](#)

[Session -50](#)

1. Spring Boot and Microservices: A Mandate for Java Developers

- Spring Boot and Microservices are **mandatory skills** for Java developers in today's industry.
- Lack of knowledge in these technologies reduces a Java developer's **resume strength by 50%**.
- Critical for **career survival and progression** in Java development.

2. The Monolithic Application Problem

- Historically, most applications were **monolithic** (single, large applications serving all functionalities).
- Common across domains: **finance, transportation, healthcare, insurance, telecom**.
- Major drawback: **Single point of failure** — one broken functionality affects the entire system.
- Example: **IRCTC reservation system** — if registration fails, search and booking also fail.
- In banking, failure in **fund transfer** can also disrupt **withdrawals and balance checks**, causing **profit and reputation losses**.

3. Introduction to Microservices: The Solution

- **Microservices architecture** breaks a large monolithic application into **smaller, independent services**.
- Each microservice handles **a specific functionality** and runs independently.
- Advantage: Failure of one microservice **does not affect the entire system**.
- No strict limit on number of microservices — real-world projects may use **dozens (e.g., 74)**.

4. The Role of Spring Boot in Microservices Development

- **Spring Boot API is mandatory** for building microservices.
- Without Spring Boot knowledge, microservices development is not feasible.
- Replaces the older **Spring Framework**, which is considered outdated.
- Key features of Spring Boot that simplify development:
 - Zero configuration
 - Starters
 - Profiles

- Embedded server
- Activators

5. Enhancing Microservices with Spring Cloud

- Microservices introduce **new challenges** (coordination, communication, reliability).
- **Spring Cloud** provides solutions by adding essential features **on top of microservices**.
- Helps manage, secure, and integrate distributed microservices effectively.

6. Recommended Learning Path & Real-Time Architecture

- Step 1: Learn **Spring Boot API** thoroughly.
- Step 2: Develop expertise in **Microservices development**.
- Step 3: Add **Spring Cloud** to enhance microservices.
- This sequence reflects **real-world architecture** in domains like finance, healthcare, and insurance.

7. Interview Relevance and Domain Knowledge

- Beyond technical expertise, interviewers expect:
 - **Domain knowledge** (finance, healthcare, insurance, etc.).
 - Clear explanation of **roles, responsibilities, and contributions** in past projects.
 - Ability to discuss **project details, client background, and implemented scenarios**.
- Demonstrates **practical application** of Spring Boot and Microservices beyond theory.

I. The Indispensable Role of Spring Boot and Microservices

- Spring Boot and Microservices are **mandatory skills** for modern Java developers.
- Industry projects (both migrations and new builds) **always use Spring Boot + Microservices**.
- Without these skills, a Java developer **cannot survive in the industry**.
- Excluding Spring Boot and Microservices from a resume **cuts its value by 50%**.
- Strong market demand creates a **critical skill gap** for those without knowledge.

II. Core Advantages of Spring Boot

A. Zero Configuration

- Eliminates extensive **boilerplate configuration** required in the Spring Framework.
- Developer focus shifts to **business logic/Java bean implementation**, not config files.
- **Automatic configuration** handled by Spring Boot.
- Spring Framework required:
 - XML or annotation-based **bean definitions**.
 - Manual **container setup** (BeanFactory, ApplicationContext).
 - Lengthy **database connection configuration** (driver, URL, credentials).
 - Dispatcher servlet, view resolvers, and handler mappings.
- Spring Boot removes these steps, streamlining development.

B. Starters (Dependency Management)

- **Starters = collections of JARs** packaged with compatible versions.
- Simplifies Maven/Gradle **dependency management**.
- Spring Framework required manual addition of JARs → frequent **version conflicts**.
- Example: upgrading JSTL versions might break compatibility in Spring Framework.
- Spring Boot Starters (e.g., spring-boot-starter-web, spring-boot-starter-jdbc, spring-boot-starter-data-jpa) guarantee **100% version compatibility**.
- Reduces **configuration headaches** and accelerates development.

C. Embedded Servers

- Spring Boot provides an **embedded server** (Tomcat/Jetty/Undertow) bundled with the app.
- Traditional approach: required **external server installation** and manual deployment.
- Benefits:
 - Eliminates **external deployment steps**.
 - Prevents **compatibility issues** between app and server.
 - Ensures **consistency** across dev, test, and production.

D. Profiles Concept

- Profiles allow **environment-specific configuration** without touching code.
- Example:
 - Dev → Oracle DB.
 - QA → MySQL DB.
- Real-world apps may have **5–6 environments** (SIT, UAT, Pre-prod, Live, etc.).
- Spring Boot simplifies profile activation (just select the profile, no code changes).
- Automatically switches to the correct environment configuration.

E. Actuators

- Provide **production-ready monitoring and management** tools.
- Expose runtime details:
 - Application health (up or down).
 - Available endpoints.
 - JVM memory usage.
 - Active threads.
- Mainly used by **production/ops teams**, but developers must **configure and enable them**.

I. Minimum Requirements for Spring Boot Application Development

- **JDK 1.8 (preferred)** – still the most widely used in industry despite newer versions (17, 21, 23).
- **STS (Spring Tool Suite)** – IDE similar to Eclipse, widely used for Spring development.
- **Maven Tool** – essential for project creation, compilation, and build management.
- Industry recommendation: stick to **JDK 1.8 + Maven 3.x** for stability and compatibility.

II. Verifying a Newly Created Spring Boot Project**1. pom.xml File**

- Must contain **parent project**:
- `<parent>`
- `<groupId>org.springframework.boot</groupId>`
- `<artifactId>spring-boot-starter-parent</artifactId>`
- `<version>2.x</version><!-- e.g., 2.7.12 -->`
- `</parent>`
- Dependencies:
- `<dependency>`
- `<groupId>org.springframework.boot</groupId>`
- `<artifactId>spring-boot-starter</artifactId>`
- `</dependency>`
- Maven Plugin:
- `<plugin>`
- `<groupId>org.springframework.boot</groupId>`
- `<artifactId>spring-boot-maven-plugin</artifactId>`
- `</plugin>`
- Ensure **JDK 1.8** and **Maven 3.x** are set in pom.xml properties.

2. Spring Boot Main Class

- Auto-generated but must be verified:
- `@SpringBootApplication`

- public class DemoApplication {
- public static void main(String[] args) {
- SpringApplication.run(DemoApplication.class, args);
- }
- }
- }
- Acts as the **entry point** for running the application.

III. Creating a Basic Spring Component (Bean)

- Create a **package** (e.g., com.durasoft.beans) and a **Java class** (e.g., HelloBean).
- Annotate with **@Component** to make it a Spring-managed bean.
- **@Component** transforms an ordinary Java class into a **Spring bean/component**.

IV. Important Spring Boot Annotations Categorization

1. Stereotype Annotations (Class-Level)

- **@Component** → Super-annotation; converts class into Spring component.
- **@Service** → Marks business logic layer (processing data, services).
- **@Repository** → Marks DAO/persistence layer (database operations).
- **@Controller** → Handles web requests in MVC applications.
- **@RestController** → Specialized controller for REST APIs.
- Note: **@Service**, **@Repository**, **@Controller**, and **@RestController** are **specialized subtypes of @Component**.

2. Autowire Annotations (Dependency Injection)

- **@Autowired** → Injects one bean into another automatically.
 - Default required=true; if false, injection becomes optional.
- **@Qualifier** → Resolves ambiguity when multiple implementations exist for the same type.
- Together they enable **dependency injection (DI)** in Spring Boot.

3. Miscellaneous Annotations

- Request Mapping & HTTP method mappings:
 - @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping.
- Validation & Data Binding:
 - @Valid, @ModelAttribute, @RequestParam, @PathVariable.
- Mandatory for **Spring MVC, REST APIs, Microservices, and Cloud apps**.

✓ **Key Takeaway:** A solid grasp of **core Java + annotations** is essential before moving into advanced Spring Boot, Microservices, and Cloud development.

I. Introduction to Spring Framework

- Provides **lightweight Java components** for efficient application development.
- **No server-dependent XML files** required → simplifies configuration.
- **Simplified testing**: uses Spring containers instead of external servers (unlike EJB).
- Key modules: **IOC, JDBC, AOP, MVC**.
- Current session focus: **Spring IOC module**.

II. Inversion of Control (IOC)

- **Definition**: Object creation/management responsibility is inverted → handled by Spring Framework, not the programmer.
- **Programmer's role**: Build **business logic classes (components)**.
- **Spring Framework's role**:
 - Create objects.
 - Initialize values.
 - Destroy objects.
- **Essence**: "Programmer defines what, Spring defines how."
- **Containers**: Java classes that manage object lifecycle.

III. Dependency Injection (DI)

- **Definition**: Mechanism where Spring provides required objects (dependencies) to application components.
- **Relationship with IOC**: DI is the technique used to achieve IOC.
- **Benefits**:
 - Better **performance** (object reuse, reduced memory usage).
 - Less **boilerplate code** (no need for new operators).
- **Injection types**:
 1. **Setter Injection** – via setters in XML <property> (demo focus).
 2. **Constructor Injection** – via constructor parameters.

3. **Interface Injection** – briefly mentioned.
4. **Lookup Injection** – briefly mentioned.

IV. Practical Implementation: XML-Based Configuration

1. **Define Java Bean** (e.g., WelcomeBean with message property & setter).
2. **XML Config (spconfig.xml):**
 - Root <beans> element.
 - <bean> element with id + class.
 - <property> for setter injection with <value>.
3. <bean id="ID1" class="com.bin.WelcomeBean">
4. <property name="message">
5. <value>Welcome to spring framework</value>
6. </property>
7. </bean>
8. **Client App (SpringClient):**
 - Initialize container → ClassPathXmlApplicationContext.
 - Retrieve object → ctx.getBean("ID1").
 - Use bean → call business methods (e.g., wb.show()).
9. **DTD/XSD** required for XML schema validation.

V. Spring Containers

- **Basic container:** BeanFactory (older, Spring 1.x).
- **Advanced container:** ApplicationContext (preferred in modern Spring 5.x).
 - Implementations:
 - ClassPathXmlApplicationContext – XML in classpath.
 - FileSystemXmlApplicationContext – XML in file system.
 - XmlWebApplicationContext – for web apps.
 - AnnotationConfigApplicationContext – annotation-only apps.

VI. Setting Up the Spring Environment

- Required JARs for Spring IOC:
 - spring-beans.jar
 - spring-context.jar
 - spring-core.jar
 - spring-expression.jar
 - jcl-over-slf4j.jar (or commons-logging.jar)
 - Recommended: **Spring 5.x (e.g., 5.2.3 release)**.
-

VII. Transition to Spring Boot and Annotations

- XML is important for **foundational understanding**.
 - Spring Boot replaces XML with **annotation-based configuration**.
 - Boot development: "Annotation A to Z" → no XML needed.
-

VIII. Key Takeaways and Advice

- **Understand IOC + DI fundamentals** with XML before moving to Spring Boot.
- Spring container manages **object lifecycle** → boosts performance & productivity.
- **DI = Practical method of IOC** (container provides objects).
- **Trust the container**: Delegate object creation/management to Spring.
- **Hands-on practice** is essential for mastery.

1. Introduction to Spring Framework and Injection Mechanisms

- Spring provides multiple dependency injection (DI) mechanisms:
 - **Setter Injection** (focus of this discussion)
 - Constructor Injection
 - Lookup Injection
 - Interface Injection
- In practice, **setter injection is most commonly used** in IT projects.

2. Types of Setter Injection

- **Primitive / String-based Dependency:** Injects simple values (e.g., int, String).
- **Object-based Dependency:** Injects one bean into another (inner → outer bean).
- **Collection-based Dependency:** Injects sets, lists, maps, or properties into a bean.

3. Inner Bean and Outer Bean Concepts

- **Outer Bean (Dependent):** Bean that depends on another (e.g., SampleBean).
- **Inner Bean (Collaborator):** Bean injected into the outer bean (e.g., DemoBean).
- **Rule:** Spring always creates the inner bean **first**, then the outer bean.

4. Implementing Setter Injection (Object-based)

- **Requirement:** Bean class must define a setter method for each property.
- **XML Config (basic example):**
 - `<bean id="ID1" class="com.da.beans.SampleBean">`
 - `<property name="db">`
 - `<bean class="com.da.beans.DemoBean"/>`
 - `</property>`
 - `</bean>`
- **Retrieving from container:**
 - `ApplicationContext ctx = new ClassPathXmlApplicationContext("SPConfig.xml");`

- `SampleBean sb = (SampleBean) ctx.getBean("ID1");`
- `sb.show();`

5. Problems with Direct Inner Bean Configuration

- **Cannot retrieve inner bean separately** (only available through outer bean).
- **Memory overhead:** Same inner bean injected into multiple outer beans creates multiple instances → higher memory usage → reduced performance.

6. Solution: Separate Inner Bean Configuration

- Define inner bean independently with its own ID.
- Reference it in outer beans using ref.
- **Improved XML Config:**
- `<bean id="ID3" class="com.da.beans.DemoBean"/>`
- `<bean id="ID1" class="com.da.beans.SampleBean">`
- `<property name="db" ref="ID3"/>`
- `</bean>`
- `<bean id="ID2" class="com.da.beans.ExampleBean">`
- `<property name="db" ref="ID3"/>`
- `</bean>`
- **Benefits:**
 - Single instance (singleton by default).
 - Saves memory.
 - Inner bean (DemoBean) can be retrieved directly by ID.

7. Dependency Injection with Collections

- **Supported Collections:**
 - **Set:** No duplicates, unordered. (`<set>`)
 - **List:** Allows duplicates, preserves order. (`<list>`)
 - **Map:** Key-value pairs. (`<map>` + `<entry>`)

- **Properties:** Key-value pairs, values optional. (<props> + <prop>)
- **Internal Behavior:**
 - Set → HashSet
 - List → ArrayList
 - Map → HashMap
- **Example:**
 - <property name="employees">
 - <map>
 - <entry key="K1" value="100"/>
 - <entry key="K2" value="200"/>
 - </map>
 - </property>
 - Spring auto-converts values (e.g., "100" → Integer).

8. Important Notes & Best Practices

- **IOC Module is the foundation:** Must master it before higher-level modules (e.g., Spring Boot).
- **Setter methods are mandatory** for setter injection to work.
- **XML vs. Annotations:** XML is used for learning, but in Spring Boot, annotations dominate.
- **Dynamic Values:** Possible (covered in Spring JDBC).
- **Learning Path:** Learn Spring Core first → then move to Spring Boot. Skipping Spring Core leads to weak understanding.

✓ Key Takeaways:

- Setter injection is flexible, resolves circular dependency issues, and supports primitive, object, and collection dependencies.
- Avoid inline inner bean definitions → prefer separate bean configuration with IDs for reusability and memory efficiency.
- Collections (List, Set, Map, Properties) are fully supported in DI.
- XML is foundational, but annotation-based configuration is the industry standard in Spring Boot.

I. Dependency Injection Mechanisms in Spring**A. Constructor Injection**

- **Definition:** Dependencies are provided via a parameterized constructor.
- **Implementation:**
 - Define constructor in bean class.
 - Configure using `<constructor-arg>` in XML.
 - Example:
 - `<bean id="ID1" class="com.org.Beans.demo">`
 - `<constructor-arg value="Welcome to constructor injection"/>`
 - `</bean>`
- **How it works:** Container calls the constructor and passes arguments at object creation.
- **Problems:**
 - **Multiple Constructors:** Each requires a separate `<bean>` config → multiple objects created → high memory consumption.
 - **Circular Dependency:** If Class A needs Class B, and Class B needs Class A → unresolvable deadlock.
 - Constructor injection **cannot resolve circular dependencies**.

B. Setter Injection

- **Definition:** Dependencies are provided via setter methods after object creation.
- **Implementation:**
 - Define setter methods in bean class.
 - Configure using `<property>` in XML.
 - Example:
 - `<property name="message" value="Welcome to setter injection"/>`
- **Behavior:** If both constructor and setter injection are applied → setter injection overrides constructor injection.
- **Advantages:**

- **Resolves Circular Dependency:** Container creates empty objects first, then injects dependencies via setters.
- **Spring Version Note:**
 - Up to Spring 5.1 → mix of constructor + setter worked.
 - From Spring 5.3.2 → both classes must use setters to resolve circular dependency.
- **Recommendation:** Prefer setter injection to avoid memory overhead and circular dependency issues.
- **Industry Usage:** Setter injection is most commonly used in real-time projects.

II. Maven as a Build Tool

- **Definition:** Maven is a build tool used for rapid application development and project delivery.
- **Key Features & Advantages:**
 - **Dependency Management:**
 - Declare dependencies in pom.xml.
 - Maven downloads required JARs automatically (no manual JAR management).
 - **Build Lifecycle Management:**
 - **clean** → removes old generated files.
 - **compile** → compiles source code.
 - **package** → creates deployable JAR/WAR.
 - **Rapid Development:** Automates builds, reducing manual effort.
 - **Jar File Creation:** Developers can package applications into JARs for client execution.
 - **Version Management:** Updating dependency versions in pom.xml auto-downloads the correct library from central repositories.
- **Spring Boot Integration:**
 - All Spring Boot projects are **Maven-based by default**.

III. Future Topics

- **Interface / Lookup Injection** → injecting via interfaces.
- **Bean Lifecycle** → stages of a bean in the container.

- **Annotation-Based Configuration** → annotations fully replacing XML in modern Spring Boot applications.

✓ **Key Takeaways:**

- **Constructor Injection** → powerful but problematic (memory overhead, unresolvable circular dependencies).
- **Setter Injection** → more flexible, resolves circular dependency, widely used.
- **Maven** → essential build tool in Spring Boot, handles dependencies, builds, and versioning automatically.
- **Future Direction** → move away from XML configs towards annotations and modern Spring practices.

◆ 1. Introduction to Injection Mechanisms

- Spring Framework supports multiple dependency injection mechanisms.
- **Setter Injection & Constructor Injection:** Used in 99% of real-world cases.
- **Interface Injection & Lookup Injection:** Rarely used, but useful in special scenarios.
- Purpose: Achieve **loose coupling, modularity**, and container-managed object creation.

◆ 2. Interface Injection

- **Definition:** Container calls a method from a special interface to inject dependencies.
- **Rarity:** Almost never used in practice; setter injection is preferred.
- **Purpose:** Avoids manual new operator usage; lets Spring container provide objects.
- **Spring-Provided Interfaces:**
 - **BeanNameAware** → Provides bean ID from Spring container.
 - Method: `setBeanName(String name)`
 - Example: Injects `cashPayment` ID into a `SampleBean`.
 - **BeanFactoryAware** → Provides `BeanFactory` (basic Spring container).
 - Method: `setBeanFactory(BeansFactory factory)`
 - Usage: Access beans programmatically.
 - **ApplicationContextAware** → Provides `ApplicationContext` (advanced container).
 - Method: `setApplicationContext(ApplicationContext ctx)`
 - Usage: Access beans, resources, and events without manually creating context.
- **When to Use:**
 - Rare cases when bean needs access to:
 - Its own bean name (`BeanNameAware`)
 - The `BeanFactory` object (`BeanFactoryAware`)
 - The `ApplicationContext` object (`ApplicationContextAware`)

◆ 3. Lookup Injection

- **Definition:** Allows singleton beans to obtain a new instance of a prototype bean whenever needed.
- **Problem Solved:** Prevents singleton beans from always using the same prototype instance.
- **Analogy:**
 - **ATM Machine (Singleton)** interacts with **Bank Server (Prototype)** for new requests.
- **Implementation Rules:**
 1. The dependent bean class must be **abstract**.
 2. It should define an **abstract method** returning the prototype bean.
 3. Spring configuration maps the method to a prototype bean using <lookup-method>.
- **Example:**
 - A1 (singleton, abstract class) has createInstance() method.
 - B1 (prototype bean) provides processPayment().
 - Spring maps createInstance() → returns new B1 each time.
- **Key Fact:** Must declare prototype beans explicitly (scope="prototype") to ensure uniqueness.
- **Bean Scopes Mentioned:**
 - singleton (default), prototype, request, session, application.

◆ 4. Development Environment Tools

- **STS (Spring Tool Suite)&Eclipse** → Free, widely used.
- **IntelliJ IDEA:**
 - Ultimate Edition → Paid, with built-in Spring support.
 - Community Edition → Free, but requires manual library setup.
- **Industry Practice:** Mixed use of STS, Eclipse, IntelliJ depending on licenses and team preferences.

◆ 5. Conclusion

- **Setter & Constructor Injection** = Dominant, practical methods.
- **Interface Injection** = Rare, but useful for accessing bean name, factory, or context.
- **Lookup Injection** = Solves prototype injection issue in singleton beans.
- Mastery of these mechanisms provides **deeper insight into Spring's dependency injection capabilities**.

I. Spring Bean Life Cycle

1. Phases of a Spring Bean Life Cycle

- **Doesn't Exist:** Bean object not yet created.
- **Instantiation:** When requested (`ctx.getBean()`), memory is allocated.
- **Initialization:** Bean properties set (like entering PIN after inserting ATM card).
- **Service Provision:** Bean executes its business methods.
- **Destruction:** Bean object destroyed when no longer needed.
- **Doesn't Exist (Again):** Returns to non-existent state.
- **Quote:** "If any bean having these steps... such a type of beans we call them as spring bean."

2. Programmatic Life Cycle Methods

- **Interfaces provided by Spring:**
 - **InitializingBean:** Defines `afterPropertiesSet()` → called after all properties set.
 - **DisposableBean:** Defines `destroy()` → called before destruction.
- **Container Control:**
 - To trigger `destroy()`, app must use `ConfigurableApplicationContext` and call `close()`.
 - **Quote:** "You need to close the container, then only destroy method is going to be executed."

3. Declarative/User-Defined Life Cycle Methods

- **Custom init/destroy methods:** Defined by developers.
- **Configured via XML:** Use `init-method` and `destroy-method` attributes in bean definitions.
- **Quote:** "We can inform container [about] init method and destroy method."

II. Annotations in Spring & Spring Boot

1. General Rules

- All annotations begin with `@`.
- Start with capital letter (e.g., `@Component`).
- Can accept inputs/attributes (e.g., `@Table(name="employee_table")`).

2. Categories of Annotations

A. Stereotype Annotations

- **@Component:** Makes a class a Spring-managed component.
- **Sub-annotations (specialized roles):**
 - **@Service:** Business logic layer.
 - **@Repository:** Persistence/data access layer.
 - **@Controller / @RestController:** Web layer (MVC/REST).
- **Key Point:** All inherit from @Component, so are Spring beans by default.
- **Retrieving Beans:** Default ID = class name with lowercase first letter; can override with @Service("customName").
- **@ComponentScan:** Instructs Spring to scan base packages for annotated components.
 - Usually combined with @Configuration in config classes.

B. Autowired Annotations

- **@Autowired:** Injects one class object into another (field, setter, constructor).
 - required=true (default): Bean must exist.
 - required=false: Bean optional.
- **@Qualifier:** Resolves conflicts when multiple beans of same type exist.
 - Specifies exact bean ID/name to inject.
 - Example: Interface UserInterface with implementations UserOne and UserTwo.

C. Miscellaneous Annotations

- **@Scope:** Defines bean scope (singleton, prototype, request, session).
 - Default = singleton.
- **@Value:** Injects values from properties files.
 - Example: @Value("\${db.user}") → reads from user.properties.
 - Needs @PropertySource to load file.
- **@Configuration:** Marks a class as Spring configuration (alternative to XML).
 - Often used with @Bean methods + @ComponentScan.
- **@PropertySource:** Loads external properties file into Spring Environment.
 - Example: @PropertySource("classpath:user.properties").

1. Executive Summary

- Focus on **building Spring JDBC applications** using the **DAO (Data Access Object) design pattern**.
- Goal: Achieve **loosely coupled architecture** by separating business logic from data access logic.
- **Relevance:**
 - Spring JDBC is **foundational** and useful for maintaining older projects.
 - **New projects** mainly use **Spring Data JPA with Spring Boot**.

2. Loose Coupling vs. Tight Coupling

- **Problem:** Early designs had both business logic and data access in one class → **tightly coupled**, hard to maintain.
- **Solution:** Introduce **loosely coupled components**, where each layer has a clear responsibility.

3. DAO (Data Access Object) Design Pattern

- **Purpose:** Achieves loose coupling by separating data access logic from business logic.
- **Structure:**
 - **DAO Interface (e.g., EmployeeDAO):** Defines abstract methods (insertEmployee, selectAllEmployees, updateEmployee).
 - **DAO Implementation (e.g., EmployeeDAOImpl):** Contains database communication logic.
 - **POJO/DTO (e.g., Employee):** Simple Java object with properties (empNumber, eName, salary, deptNumber).
 - **Business Component (e.g., EmployeeBean):** Contains business rules and calls DAO methods but **does not handle persistence logic**.

4. Core Components in Spring JDBC Application

- **Employee (POJO/DTO):** Properties + getters/setters.
- **EmployeeDAO (Interface):** Declares methods for CRUD operations.
- **EmployeeDAOImpl (Implementation):**
 - Uses **JdbcTemplate (JT)** injected via **setter injection**.
 - Example insert:

- Extract values from Employee object.
- Store in Object[] params.
- Execute with JT.update("insert into emp values (?, ?, ?, ?)", params).
- Example select:
 - JT.queryForList("select * from emp") → iterate with Iterator.
- **Method usage guidelines:**
 - **DDL:** execute()
 - **DML (Insert/Update/Delete):** update()
 - **Queries:** query(), queryForList()
- **DAO Design:** Typically one DAO interface per table.
- **EmployeeBean (Business Logic):**
 - Holds a dependency on **EmployeeDAO** (injected via setter).
 - Example: insert(Employee e) → internally calls dao.insertEmployee(e).

5. Configuration & Client Setup

- **Configuration (XML-based):**
 - Define **DriverManagerDataSource**.
 - Define **JdbcTemplate**.
 - Define **EmployeeDAOImpl** (inject JdbcTemplate).
 - Define **EmployeeBean** (inject DAO).
- **Client/Test Class:**
 - Load context with ClassPathXmlApplicationContext.
 - Get EmployeeBean from context.
 - Create Employee object, set properties, call methods.
- **Flow:** Client → EmployeeBean → EmployeeDAO → Database.

6. Improving Data Display with RowMapper

- **Problem:** queryForList() returns key-value pairs, less readable.
- **Solution:**

- Implement **RowMapper<Employee>**.
- Override `mapRow(ResultSet rs, int rowNum)` → populate Employee object.
- Use with `JT.query("select * from employee", new EmployeeMapper())`.
- **Benefit:** Converts results directly into **object format** for clean display.

7. Important Facts & Recommendations

- **Industry Practice:** Spring JDBC is legacy; **Spring Data JPA + Boot** is standard for new apps.
- **Setter Injection:** Preferred in examples (`setJt`, `setDao`).
- **Dynamic SQL:** Use parameterized queries (?) for flexibility & security.
- **Error Handling:** Must handle invalid credentials, SQL issues, etc.
- **DAO Best Practice:** One DAO per table.
- **Learning Support:** Review earlier sessions for **DDL/DML/query decisions**.

8. Conclusion

- DAO pattern ensures **loose coupling** and clean separation of concerns.
- Covers **all layers**: POJOs, DAO interface, DAO implementation, business component, configuration, and client.
- Enhancements like **RowMapper** improve usability.
- **Current industry shift:** From Spring JDBC → Spring Data JPA with Boot, but JDBC knowledge is vital for **foundational understanding** and **legacy system maintenance**.

I. Introduction to Spring Boot and Microservices

- Spring Boot = Framework on top of **Spring Framework**.
- Goal: Simplifies Spring app development → especially useful for **microservices**.
- Addresses shortcomings of traditional Spring (no starters, embedded servers, or zero-configuration).
- Provides **faster development** and **reduced boilerplate code**.
- Example project: Display message "Welcome to Spring Boot and Microservices".

II. Key Spring Components and Annotations

A. @Component Annotation

- Marks a **Java class** as a Spring-managed component.
- Enables Spring container to manage lifecycle + allow dependency injection.

B. CommandLineRunner Interface

- Executes logic immediately **after Spring context loads** and before app run completes.
- **Implementation steps:**
 - Implement CommandLineRunner.
 - Override public void run(String... args).
 - Annotate with @Component for Spring Boot to detect.
- **Execution order control:**
 - Use @Order annotation.
 - Lower value = higher priority. (e.g., @Order(1) runs before @Order(2)).

C. @Autowired Annotation

- Enables **automatic dependency injection**.
- Eliminates manual object creation with new.
- Example:
 - @Autowired HelloBean bean;
 - bean.sayHello();
- Prevents NullPointerException by ensuring Spring provides bean instances.

III. The Core SpringApplication Class & Execution Flow

- **SpringApplication** → central class for running Spring Boot apps.
- Even though Boot app, **internally runs as a Spring application**.

Execution flow of SpringApplication.run():

1. Creates **ApplicationContext** (Spring container).
2. Registers **command-line property source** (reads CLI arguments).
3. Refreshes **ApplicationContext** → loads all **singleton beans**.
 - Singleton = one shared instance per application context (e.g., ATM common data).
 - Built using: private constructor + static variable + static method.
4. Executes **CommandLineRunner beans** (if available).
 - Execution order follows @Order annotations.

IV. The @SpringBootApplication Annotation

- A **meta-annotation** combining three key annotations:
1. **@SpringBootConfiguration**
 - Marks class as Spring Boot configuration.
 - Equivalent to @Configuration in traditional Spring.
 - Modern apps use **annotation-based configs** (instead of XML).
 2. **@ComponentScan**
 - Scans **base package and subpackages** for annotated Spring components.
 - Auto-detects @Component, @Service, @Repository, @Controller.
 - Removes need for manual new object creation.
 3. **@EnableAutoConfiguration**
 - Enables **auto-configured Spring Boot components**.
 - Saves effort by creating defaults (e.g., DataSource, JdbcTemplate).
 - Reduces manual configuration → faster dev cycles.

Summary / Key Takeaways

- Spring Boot = lightweight, **fast**, and **config-minimized** approach for Spring apps.

- Core elements:
 - @Component → registers beans.
 - CommandLineRunner → executes startup logic.
 - @Autowired → dependency injection.
 - SpringApplication.run() → manages lifecycle.
 - @SpringBootApplication → unifies configuration, scanning, and auto-setup.
- Industry standard for **microservices** and **modern Spring projects**.

1. Purpose and Importance

- Command line arguments allow **dynamic configuration at runtime**.
- Use cases:
 - **Runtime Server Configuration:** Run servers on different ports for load balancing.
 - **Microservices Control:** Pass configs to different microservice instances.

2. Types of Arguments

- **Optional Arguments (Key-Value Pairs)**
 - Syntax: -E=value or -first name=duda
- **Non-Optional Arguments (Values Only)**
 - Syntax: value1 or amir

3. Syntax for Execution

- General form:
- `java -jar target/filename.jar -key1=value1 valueA -key2=value2 valueB`
- Example:
- `java -jar target/filename.jar -first name=duda amir -last name=tex hydra`

4. Application Development Workflow

4.1 Project Creation

- Create **Spring Starter Project** (e.g., *option and non option app*).
- Use **Maven Project, JDK 1.8, and Spring Boot 2.x**.
- Default spring-boot-starter-parent dependency is enough.

4.2 Configuration & Setup

- Verify pom.xml (JDK + Boot versions).
- Run **Maven update** to refresh dependencies.
- SpringBootClass is auto-generated in base package.

4.3 Argument Reading Logic

- Create package com.g.g.bins.
- Define **bean class** (e.g., OptionAndNonOptionB).
- Annotate with @Component.
- Use **ApplicationArguments interface**:
 - args.getNonOptionArgs() → list of non-optional values.
 - args.getOptionNames() → get keys of optional args.
 - args.getOptionValues(key) → get values for each key.
- Print using **Java 8 features**:
- nonOptionList.forEach(str -> System.out.println("Non-Optional Value: " + str));

4.4 Runner Class

- Create package com.g.g.runner.
- Define MyRunner implementing CommandLineRunner.
- Annotate with @Component.
- Autowire bean (OptionAndNonOptionB) and call showArgs() inside run().

5. Packaging & Execution

5.1 Create JAR File

- Run Maven build → Goals: clean package.
- Handles:
 - clean = remove old builds.
 - package = compile + generate JAR.
- **Error fix:** If "no compiler provided" → project is using JRE, switch to **JDK** in project settings.

5.2 Run JAR with Arguments

- Navigate to project root in CMD.
- Execute:
 - java -jar target/optionandnonoptionapp-1.0.jar -first name=gadura amir -last name=tex hydra

6. Future Topics

- **Spring Boot JDBC** → Persistence logic with DB.

- Spring Boot MVC → Web layer, after JDBC.

7. Important Development Practices

- **Standard Project Structure** → DAO, Service, Controller packages.
- **JDK 1.8 Features** → lambdas, forEach for cleaner code.
- **Self-Troubleshooting** → Analyze & fix issues independently (real-world expectation).

-
- ☑ This covers **why, how, and step-by-step** execution of passing command line arguments in Spring Boot.

1. Introduction to Spring Boot JDBC & Persistency

- Spring Boot JDBC enables **persistency logic**: connecting to a database and performing CRUD operations.
- Industry trend: **Spring Boot Data JPA** is preferred; JDBC is foundational knowledge.
- Key points:
 - JDBC is still relevant for understanding Data JPA.
 - JDBC provides hands-on understanding of SQL operations within Spring Boot.

2. Persistency Operations & JDBC Template

- **CRUD Operations**: Create, Read, Update, Delete.
- **JDBC Template**: Core class for executing database operations with predefined methods.

2.1 JDBC Template Methods & SQL Types

- **execute()** → DDL (CREATE, DROP, ALTER, TRUNCATE, RENAME)
- **update()** → DML (INSERT, UPDATE, DELETE)
- **query methods** (queryForInt, queryForList, queryForObject) → DQL (SELECT)

Developer responsibilities:

- Java developers handle DDL, DML, DQL.
- DBAs handle DCL (GRANT, REVOKE) and TCL (COMMIT, ROLLBACK).
- Modern developers expected to have **full-stack knowledge**.

3. Developing a Spring Boot JDBC Application: Practical Steps**3.1 Required Dependencies**

- spring-boot-starter-jdbc → JDBC template & components
- mysql-connector-java → MySQL connectivity

3.2 Project Setup

- Create a Spring Boot project.
- **Java 1.8** recommended (Spring Boot 2.x).
- Add above dependencies.

3.3 Application Properties (application.properties)

- `spring.datasource.driver-class-name` → e.g., `com.mysql.cj.jdbc.Driver`
- `spring.datasource.url` → e.g., `jdbc:mysql://localhost:3306/test`
- `spring.datasource.username` → e.g., `root`
- `spring.datasource.password` → e.g., `root`

3.4 Code Structure & Annotations

- **Repository Layer**
 - Interface (e.g., `ITest`) defines DB operations.
 - Implementation class (e.g., `TestImpl`) annotated with `@Repository`.
 - Inject `JdbcTemplate` via `@Autowired`.
- **Service / Business Logic Layer**
 - Test class (e.g., `TestJdbc`) annotated with `@Component`.
 - Inject repository interface; call methods like `insertRecords()`, `selectRecords()`.

3.5 Performing Database Operations

- **Insert:** `jdbcTemplate.update("INSERT INTO emp VALUES (?, ?, ?, ?)", ...)`
- **Select:** `jdbcTemplate.queryForList("SELECT * FROM emp")` → returns a List

3.6 Functional Programming with Java 1.8

- Use **lambda expressions** & `forEach` for concise code:
 - Reduces verbosity
 - Enables functional-style operations

3.7 Running the Application

- Run the main Spring Boot class.
- Two-tier application: Java app → Database.
- Verify database changes directly in MySQL Workbench.

4. Transition to Spring Boot Data JPA

- JDBC limitations:
 - Manual SQL queries required
 - Need to verify database updates manually

SPRING BOOT & MICRO SERVICES

- Spring Boot Data JPA automates queries & persistence.
- Instructor will cover Data JPA after MVC topics.

Key takeaway:

- JDBC knowledge is foundational; Data JPA addresses JDBC limitations for modern applications.

1. Introduction: The Need for MVC

- Pre-MVC issues:
 - No standardized request flow → inconsistent development.
 - High dependency on original developers → onboarding challenges.
 - Tightly coupled components → changes in one place affect multiple areas.
- Example: Changing a List to a Set requires updates in both JSP and Java Beans.
- MVC provides: **standardization, modularity, and loose coupling.**

2. Core Concepts of MVC

- **Model:** Application data & business logic (Service + Data Access layers).
- **View:** Presentation layer (JSP, HTML, Angular, etc.).
- **Controller:** Intermediary handling requests, interacting with Model, and selecting Views.
- Controller is considered the **heart of the application.**

3. Spring Boot MVC Request Flow

3.1 High-Level MVC Flow

1. Client sends request.
 2. Controller receives request.
 3. Controller interacts with Model.
 4. Model processes data.
 5. Model returns data to Controller.
 6. Controller selects appropriate View.
 7. Controller sends data to View.
 8. View renders response.
 9. Response sent back to Client.
- **No direct communication** between View and Model.

3.2 Detailed Spring Boot Request Flow (with Dispatcher Servlet)

1. Client Request → Browser.

2. **Dispatcher Servlet (Front Controller)** → central entry point for all requests.
3. **Handler Mapping** → identifies subcontroller and handler method.
4. Subcontroller Identification → returned to Dispatcher Servlet.
5. Dispatcher Servlet invokes Subcontroller.
6. Subcontroller → Service Layer → business logic.
7. Service Layer → Data Access Layer → database operations.
8. Database returns data → Data Access Layer → Service Layer → Subcontroller.
9. Subcontroller returns **logical view name** + model data.
10. **View Resolver** → maps logical view to physical template (e.g., JSP/HTML).
11. Dispatcher Servlet renders view with model data.
12. Response sent back to Client.

4. Layering and Annotations

- **Data Access Layer** → @Repository
- **Service Layer** → @Service
- **Controller Layer** → @Controller
- **View Layer** → presentation logic (JSP/HTML)
- Annotations clarify the role of a class **without reading full implementation**.

5. Types of MVC Applications

- **MVC1**: One front controller + one subcontroller → suitable for **small applications**.
- **MVC2**: One front controller + multiple subcontrollers → suitable for **large-scale applications**, most modern apps follow MVC2.

6. Conclusion

- **Centralized Request Flow** → predictable, organized handling of requests.
- **Loose Coupling** → changes in one component minimally impact others.
- **Improved Collaboration** → easier onboarding and teamwork.
- MVC ensures **efficient, maintainable, scalable** application development.

I. Developing Subcontrollers

- **Subcontrollers:** User-defined controllers handling specific requests.
- **@Controller Annotation:** Marks a class as a Spring Boot controller.
 - Quote: "In order to make this class as the spring boot controller... use @Controller."
- **Handler Methods:** Methods within subcontrollers process specific HTTP requests (GET, POST, etc.).
- **Request Mapping Approaches:**
 - **@RequestMapping:** General-purpose, explicit URL and request method.
 - Syntax: @RequestMapping(value="/register", method=RequestMethod.GET)
 - Quote: "You have to tell explicitly what type of request it is."
 - **Specialized Annotations** (Preferred Modern Approach):
 - @GetMapping("/register"), @PostMapping("/add")
 - Implicitly define HTTP method; other types: @DeleteMapping, @PutMapping.
 - Quote: "The annotation itself is saying it is the GET request."
- **Handler Method Return Type:** Typically ModelAndView (contains model data + logical view name); can also return String.

II. Spring Boot MVC Application Flow**A. Components of the Flow**

1. **Browser (Client):** Initiates request (e.g., <http://localhost:2021/helloapp/hello>).
2. **DispatcherServlet (Front Controller):**
 - Receives all requests first.
 - Delegates to HandlerMapping, then receives logical view name from subcontroller.
 - Consults ViewResolver to construct physical view; sends rendered page back to browser.
 - Quote: "Dispatcher select is the front controller... first the request is received by the front controller."
3. **HandlerMapping:**
 - Identifies subcontroller and handler method based on request URL.
 - Guides DispatcherServlet to call the correct handler.

4. Subcontroller (User-defined Controller):

- Processes the request, executes business logic.
- Returns logical view name to DispatcherServlet.

5. ViewResolver:

- Maps logical view name to physical view (e.g., welcome.jsp).
- Returns physical view to DispatcherServlet.

6. View Page (JSP/HTML): Displays the response to the user.**B. Future Extension**

- Subcontroller typically interacts with:
 - **Service Layer** → business logic.
 - **Repository Layer** → data access operations.
 - **Database** → Oracle, MySQL, DB2.
- Quote: "Subcontroller communicates to service layer, service communicates to repository, repository communicates to database."

III. Practical Example: Basic Spring Boot MVC App**A. Project Setup**

- Spring Starter Project: e.g., Springboot MVC app
- Base Package: com.hello.controller
- Dependencies:
 - Spring Web (for web applications)
 - Tomcat Embed Jasper (for compiling JSP pages)
- Recommended versions: JDK 1.8, Maven 3.x

B. Controller Development (HelloController.java)

- Annotate with @Controller.
- Define handler method using @GetMapping("/hello").
- Return ModelAndView object with logical view name "welcome".
- Quote: "Return ModelAndView... welcome... logical name."

C. View Page Development (welcome.jsp)

- Folder: src/main/webapp/WEB-INF/views/
- Create welcome.jsp with HTML content (e.g., <h2>Welcome to Spring Boot MVC</h2>).

D. Configuration (application.properties)

- server.port=2021 → custom server port
- server.servlet.context-path=/helloapp → context path
- spring.mvc.view.prefix=/WEB-INF/views/ → view file location
- spring.mvc.view.suffix=.jsp → view file extension
- Quote: "Logical view name 'welcome' + prefix/suffix → physical view."

E. Running the Application

- Run main Spring Boot class.
- Access via browser: http://localhost:2021/helloapp/hello.

IV. Key Takeaways

- **DispatcherServlet** → front controller, central entry point.
- **HandlerMapping** → maps requests to controller methods.
- **ViewResolver** → maps logical view names to physical files.
- Use **@Controller** for subcontrollers.
- Prefer **@GetMapping/@PostMapping** over **@RequestMapping**.
- **JSP support** requires Tomcat Embed Jasper dependency.
- **application.properties** configures server port, context path, and view resolver settings.
- Proper folder paths and prefixes/suffixes are critical to avoid errors.
- **Standard communication pattern**: Controller → Service → Repository → Database.

Session -15

I. Project Setup and Dependencies

- **Project Name:** CRM customer registration module
- **IDE:** Spring Tool Suite (STS) → Start Project
- **Base Package:** com.CRM
- **Core Dependencies:**
 - Spring Web → Web application development
 - JDBC → Database connectivity
 - Validation → Input validations
- **Optional/Manual Dependencies:**
 - Mail (limited by Gmail restrictions)
 - JSP Support → Tomcat Embed Jasper (org.apache.tomcat:tomcat-embed-jasper) and JSTL (javax.servlet:jstl:1.2)
 - MySQL → Database connectivity
- **Recommended Versions:** JDK 1.8, Spring Boot 2.x
- **Project Update:** Required after adding dependencies

II. MVC Architecture & Flow

1. Model (com.CRM.model)

- Represents data structures (DTOs) with properties
- **Validations:** @NotEmpty, @NotNull, @Email, @Pattern (e.g., ^[a-zA-Z0-9]+\$)
- **Enums Example:** Gender { MALE, FEMALE, OTHERS }
- Example Models: Customer (registration), Login

2. DAO (com.CRM.dao)

- Handles persistence and data retrieval
- Interfaces: ILoginDAO, IRegisterDAO + implementations

3. Controller (com.CRM.controller)

- Receives HTTP requests (e.g., /CRM/index)
- **View Resolution:** Returns JSP pages (index.jsp, register.jsp)
- **Passing Data:** Model.addAttribute() → accessed in JSP via \${modelAttribute}
- **Validation Handling:**
 - @Valid @ModelAttribute("customer") Customer customer triggers validation
 - Errors captured in BindingResult
 - Displayed via <form:errors path="fieldName"/>
- **Error Example:** Email exists → "Customer has already registered with the given mail ID"

III. Database Interaction (JDBC)

- **Configuration:** application.properties → driver, URL, username, password
- **DAO Implementation:** RegisterDAOImpl → saveCustomer method
- **Email Check Before Insert:** SELECT count(*) FROM customer_tab WHERE email=?
 - Count = 1 → error
 - Count = 0 → insert new record
- **Insertion:** jdbcTemplate.update("INSERT INTO customer_tab VALUES (...)")
- **Success Message:** "Registration success"
- **Manual Management:** Developer responsible for table creation, SQL queries

IV. Email Service

- Sends registration confirmation emails
- Gmail restrictions may block third-party apps
- Configured via application.properties (host, port, username, password)

V. Demonstration Workflow

1. Start Spring Boot server → connect to MySQL
2. Access URL: http://localhost:2121/CRM/index
3. **Sign Up:**
 - Empty form → validation errors displayed

- Enter valid data → first name, last name, email, password, age, DOB, contact, country
 - Password validation: lowercase start + letters, digits, special chars
4. Successful registration → confirmation message
 5. Database verification → `SELECT * FROM customer_tab`

VI. Introduction to Spring Data JPA

- **Purpose:** Simplifies persistency logic (connect DB, perform CRUD)
- **Problem with JDBC:** Manual table creation, SQL writing, complex for developers
- **Solution:** Spring Data JPA abstracts SQL, reduces manual work
- Advantages to be discussed in a subsequent session

VII. Key Takeaways

- MVC Layers: Model → DAO → Controller → View
- Validations handled via annotations + `BindingResult`
- JDBC requires manual database management; Spring Data JPA simplifies it
- Email & JSP support requires additional dependencies
- Recommended: JDK 1.8 + Spring Boot 2.x

I. Data Binding: Capturing and Validating User Input**A. Data Binding Process**

- **UI Form Submission:** User enters data (username, full name, password, confirm password) and clicks submit
- **Request to Middleware:** Form submission is sent to backend middleware → captured as **data binding**
- **Backend Representation (DTO/Java Bean):**
 - Object to hold form data → called Command Class, Form Package, or DTO
 - Stored as key-value pairs
- **Binding at UI Side:**
 - `<form:form>` tag with `modelAttribute` binds UI to DTO
 - Example:
 - `<form:form action="save" method="post" modelAttribute="custoBin">`
 - `<!-- Input fields -->`
 - `</form:form>`
- **Capturing at Backend:**
 - `@ModelAttribute("custoBin")` in controller method retrieves DTO
 - Example:
 - `public void addUser(@ModelAttribute("custoBin") CustomerBin customer) { ... }`

B. Data Validation

- **Validation Rules:** Defined in DTO/Java Bean via annotations:
 - `@NotEmpty` → non-empty string
 - `@NotNull` → field cannot be null
 - `@Min(value)` / `@Max(value)` → numeric range
 - `@Size(min,max)` → string length limits
 - `@Past` → date must be in the past
- **DTO Example:**
 - `public class Customer {`

- @NotNull @NotEmpty private String username;
- @Min(12) @Max(25) private Integer age;
- @Size(min=5, max=15) private String password;
- @Past private Date dateOfBirth;
- @NotNull private Gender gender; // enum: Male, Female, Others
- }
- **Enabling Validation:** Add spring-boot-starter-validation dependency
- **Triggering Validation:** Use @Valid with @ModelAttribute in controller
- public void addUser(@Valid @ModelAttribute("custoBin") CustomerBin customer) { ... }
- **Error Messages:**
 - Defined in messages.properties → e.g., testing.username.notempty=User name is required
 - Displayed in JSP using <form:errors path="username"/>

II. Application Flow Diagram (Spring MVC)

A. Initial Request and Page Load

1. **URL Entry:** e.g., http://localhost:2121/CRM/index
2. **Dispatcher Servlet:** Receives all requests → front controller
3. **Handler Mapping:** Identifies correct subcontroller and handler method
4. **Subcontroller Execution:** Executes the handler method
5. **View Resolver:** Converts logical view name to physical view (e.g., index.jsp)
6. **Response to Browser:** Dispatcher Servlet sends the rendered page

B. User Interaction and Form Submission

1. **Hyperlink Click:** e.g., "Sign Up" → request sent to Dispatcher Servlet
2. **Handler Mapping & Subcontroller:** Dispatcher maps request to correct subcontroller → returns registration.jsp
3. **Form Completion & Submission:** User fills form → clicks submit
4. **Dispatcher Servlet & Handler Mapping:** Request routed to appropriate subcontroller
5. **Business Logic & Persistence:** Subcontroller interacts with:
 - **Service Layer:** Handles business logic

- **Repository Layer:** Handles data access
- **Database:** Stores the data

6. **Response:** Returns result page or displays validation errors if binding/validation fails

Key Takeaways

- Data binding links UI form fields to backend DTO objects
- Validation annotations ensure data integrity and can be customized via messages.properties
- Dispatcher Servlet → Handler Mapping → Subcontroller → Service → Repository → Database is the core Spring MVC request flow
- Errors in data binding/validation are automatically propagated back to the UI
- This flow ensures **robust, secure, and maintainable form handling** in Spring Boot applications

1. Spring Boot Application Setup and Dependencies

- Project initialized using **Spring Tool Suite (STS)**.
- **Base Package:** com.CRM
- **Core Dependencies:**
 - **Spring Web** → for web development
 - **JDBC** → for database connectivity
 - **Validation** → for input validation
- **Optional / Manual Dependencies:**
 - **Mail** → optional, currently restricted by Gmail
 - **Jasper & JSTL** → manually added for JSP presentation
 - **MySQL** → for database connectivity
- **Versions:** JDK 1.8, Spring Boot 2.x recommended for stability
- Project updated after adding dependencies

2. Application Architecture (MVC Pattern)

- Follows **Model-View-Controller (MVC)** architecture.
- **Model (DTO/POJO):**
 - Customer and Login models
 - Fields: firstName, lastName, email, password, age, gender (enum), dob, contactNumber, country
 - **Validation annotations:** @NotEmpty, @NotNull, @Email, @Pattern, @Min, @Max
- **DAO (Data Access Object):**
 - Interfaces (ILoginDAO, IRegisterDAO) define database operations
 - Concrete implementations handle JDBC operations
- **Controller:**
 - Handles requests, orchestrates application flow, interacts with DAO, returns views

3. Request-Response Flow and View Resolution

- Requests first reach the **controller**
 - Example: /CRM/index → returns index.jsp
 - Navigation (e.g., "Sign Up") triggers controller requests → returns register.jsp
-

4. Data Binding and Form Handling

- **@ModelAttribute** binds form data to model objects
 - Controller adds Customer object to model: `model.addAttribute("customer", customer)`
 - JSP form uses `modelAttribute="customer"` to bind input fields
 - On submission, `@ModelAttribute` + `@Valid` in controller retrieves and validates object
-

5. Server-Side Validation and Error Handling

- **@Valid** triggers validation based on model annotations
 - **BindingResult** stores validation errors
 - If `result.hasErrors()` → return to register.jsp
 - Error messages displayed with `<form:errors path="fieldName"/>`
 - Messages defined in `messages.properties` file
-

6. Database Interaction (JDBC)

- **JdbcTemplate** used for database operations
 - **Checking for existing records:** `SELECT count(*) FROM customer_tab WHERE email = ?`
 - **Error for duplicate email:** "Customer has already registered with the given mail ID. Try with another."
 - **Insertion:** `JdbcTemplate.update()` inserts new customer
 - **Developer responsibility:** manual creation of `customer_tab` table and writing SQL queries
-

7. Success and Email Notification

- Successful registration → "Registration success" message
 - Email service implemented but **non-functional** due to Gmail restrictions
-

8. application.properties Configuration

- **Server:** server.port=2021, server.servlet.context-path=/CRM
 - **Database:** driver class, URL, username, password
 - **Email properties:** host, port, username, password
-

9. Limitations of JDBC and Introduction to Spring Data JPA

- **Drawbacks of JDBC:**
 - Manual SQL query development
 - Manual database table creation
 - **Spring Data JPA:**
 - Simplifies persistence logic
 - Abstracts SQL and database management
 - Provides easier CRUD operations and reduces boilerplate code
-

10. Important Ideas/Facts

- End-to-end application development demonstrated from setup to testing
- Validation is crucial (front-end + back-end) for robust applications
- MVC pattern ensures separation of concerns (Model, View, Controller, DAO)
- JDBC requires significant manual effort; JPA offers abstraction and ease
- Real-world constraints (e.g., Gmail restrictions) affect functionality
- Use stable JDK (1.8) and Spring Boot (2.x) for industry practices
- External dependencies (Jasper, JSTL, MySQL) demonstrate flexibility
- Immediate user feedback via error messages improves usability

1. Introduction

- Spring Boot Data JPA is a **sub-project of Spring Data** developed by the Spring team.
- **Purpose:** Simplify persistence logic in Java applications.
- Reduces boilerplate compared to Spring Boot JDBC.
- Key takeaway: "The main intention of Spring Boot Data JPA is to implement persistency logic in projects."

2. Advantages of Spring Boot Data JPA

- **Reduced Implementation Burden:**
 - Developers define only **DAO interfaces (repositories)**.
 - Implementations are automatically handled by Spring Boot Data JPA.
- **Automated Database & Query Management:**
 - No manual table creation or SQL query writing needed.
- **Standardization & Industry Adoption:**
 - Widely used in modern Java projects for robustness and efficiency.

3. Core Concepts

3.1 Repository Interfaces

- Central to Spring Boot Data JPA; define database interaction methods.
- **Interface Hierarchy:**
 - CrudRepository → basic CRUD operations.
 - PagingAndSortingRepository → adds pagination & sorting.
 - JpaRepository → most used; extends both above and adds JPA-specific features.
- **Repository Development Rules:**
 - Must be an **interface**
 - Must **extend JpaRepository**
 - Annotate with @Repository
 - Specify two generics:

- Entity class type
- Primary key type
- Example: public interface CustomerRepository extends JpaRepository<Customer, Integer>

3.2 Entity Classes

- **Entity class** maps to a database table and its columns.
- **Annotations:**
 - @Entity → marks class as an entity
 - @Id → marks primary key
 - @Table(name="custom_table_name") → optional table name customization
- **Relationship with DTO:**
 - Entity → maps to database
 - DTO → transfers data between layers without direct DB interaction

4. Predefined and Customized Methods

- **Predefined JpaRepository Methods:**
 - save(), findById(), findAll(), deleteById(), deleteAll()
- **Customized/User-Defined Methods:**
 - Used when predefined methods do not meet requirements
 - **First Approach: Query by Method Name**
 - Naming convention generates queries automatically
 - Pattern: findBy<Property> or findBy<Property1>And<Property2>
 - Keywords: GreaterThan, LessThan, And, Or, etc.
 - Examples:
 - findByContact(Long contact)
 - findByCustomerName(String customerName)
 - findByCellGreaterThan(Double cell)
 - findByFirstNameOrLastName(String firstName, String lastName)
 - No implementation needed; Spring Boot Data JPA handles query execution
- Other approaches for customized queries (@Query, @NamedQuery) will be covered later

Key Takeaways

- Spring Boot Data JPA **abstracts persistence**, reducing manual SQL and database tasks.
- Repositories + Entity classes are the backbone of data access.
- Predefined methods handle most use cases; customized queries are easy with naming conventions.
- Widely adopted in industry for standardized, efficient persistence.

1. Overview of Query Creation Approaches

- **Three primary approaches** for creating customized queries:
 1. **Method Name Query** (findBy<Property>)
 - Requires strict naming conventions.
 - Less flexible due to naming constraints.
 2. **@Query Annotation (Highly Recommended)**
 - Most widely used and practical in real-world projects.
 - Allows flexible method naming and custom JPQL/SQL queries.
 3. **@NamedQuery Annotation**
 - Queries defined at the entity class level.
 - Supports only JPQL.
 - Less common than @Query annotation in modern projects.

2. Deep Dive into Approach 2: @Query Annotation**2.1 Key Principles**

- Flexible method naming; no constraints.
- Supports **JPQL** and **SQL** queries.
- Annotation placed directly above repository method.

2.2 JPQL Query Construction

- Database tables → **Entity classes**, columns → **Entity properties**.
- Queries are **case sensitive**.
- Use **aliases** for entities: Select C from Customer C.
- **Parameters:**
 - Positional: ?1, ?2
 - Named: :paramName with @Param annotation if needed.
- Example:
 - @Query("Select C from Customer C where C.firstName = ?1")
 - List<Customer> loadCustomersByFirstName(String firstName);

2.3 SQL Query Construction

- Use `nativeQuery = true` in `@Query` annotation.
- Follows standard SQL syntax; **case insensitive**.
- Parameters: positional (`?1`) or named (`:param`).
- Example:
- `@Query(value = "Select * from customer where first_name = ?1 or last_name = ?2", nativeQuery = true)`
- `List<Customer> getCustomersByFirstNameOrLastName(String firstName, String lastName);`

2.4 Non-Select (DML) Operations

- For **UPDATE/DELETE**, use both `@Query` and `@Modifying`.
- Example:
- `@Modifying`
- `@Query("update Employee E Set E.salary = ?1 where E.empNumber = ?2")`
- `void updateEmployeesByDepartmentNumber(double salary, int empNumber);`

3. Deep Dive into Approach 3: @NamedQuery Annotation

- Defined at the **entity class level** using `@NamedQuery` or `@NamedQueries`.
- Naming convention: `EntityName.methodName` (e.g., `employee.searchBySal`).
- Only supports **JPQL**.
- Repository methods reference the named query by its name.
- Example:
- `@Entity`
- `@NamedQuery(name = "employee.searchBySal", query = "Select E from Employee E where E.salary = ?1")`
- `public class Employee { ... }`

Repository method:

```
List<Employee> searchBySal(double salary);
```

4. Practical Implementation Steps (Passenger Example)

4.1 Project Setup

- Create Spring Boot project with **Spring Boot Data JPA** dependency.
- Add database driver (e.g., MySQL).
- Run Maven update to resolve dependencies.

4.2 Entity Class (Passenger)

- Package: com.jpa.entity
- Annotations: @Entity, @Id, optional @Column(length = 12)
- Fields: passengerId, firstName, lastName, email, seatNumber
- Generate getters/setters

4.3 Repository Interface (PassengerRepository)

- Package: com.jpa.repository
- Interface extends JpaRepository<Passenger, Integer>
- Annotate with @Repository
- Define custom query methods using @Query if needed

4.4 Runner Class (MyRunner)

- Package: com.jpa.runner
- Annotate with @Component and implement CommandLineRunner
- Inject repository with @Autowired
- Create and save Passenger objects in run() method

4.5 Configuration

- application.properties: database URL, username, password, and JPA properties (e.g., spring.jpa.hibernate.ddl-auto)

5. Key Takeaways & Recommendations

- **@Query Annotation is preferred** for most custom query requirements.
- Understand **JPQL vs SQL differences**, including entity/table names and case sensitivity.
- Use nativeQuery = true for native SQL queries.
- Use @Modifying for UPDATE/DELETE operations.
- Practice multiple examples to build clarity and confidence.
- Collaborate with DB specialists for complex queries.

I. Introduction

- Simplifies database interactions by automating table creation and SQL query generation.
- Reduces developer workload, allowing focus on business logic.
- Advantage over Spring Boot JDBC: framework handles all persistence tasks automatically.

II. Core Concepts

- **Passenger Entity:** Properties include passengerId, firstName, lastName, email, seatNumber.
- **Repository Interface:** Interacts with the database (e.g., PassengerReport).
- **Runner Class:** Entry point for testing database operations like save, find, and delete.

III. application.properties Configuration

- **MySQL Properties:** JDBC URL, username, password, driver class.
- **JPA Properties:** Control JPA behavior with the database.
- **spring.jpa.hibernate.ddl-auto:** Schema management
 - create: Creates tables; drops existing tables—data loss occurs.
 - update: Recommended; preserves existing data and updates schema if needed.
 - create-drop: Drops tables after session—used for development/testing.
- **spring.jpa.show-sql:** true displays generated SQL queries for debugging.

IV. Basic CRUD Operations (Predefined Methods)

- **save():** Persist an entity.
- **findById():** Retrieve single record; returns Optional<T> to avoid null pointer exceptions.
- **findAll():** Fetch all records; result in a List; iteration possible with Java 8 lambda.
- **deleteById() / deleteAll():** Delete single/all records.

V. Custom Queries: User-Defined Methods

- **Approach 1: Method Naming Convention (Derived Queries)**
 - Method name: findBy<PropertyName> (e.g., findByEmail).

SPRING BOOT & MICRO SERVICES

- Must match entity property names exactly (case-sensitive for first letter).
- Implementation automatically handled by Spring Data JPA.
- Example: Passenger findByEmail(String email);
- **Approach 2: Using @Query Annotation**
 - Supports JPQL or native SQL for flexible custom queries.
 - **JPQL Example:** @Query("Select P from Passenger where P.firstName = ?1 or P.lastName = ?2")
 - **Native SQL Example:** @Query(value = "SELECT * FROM passenger WHERE first_name = ?1", nativeQuery = true)
 - Parameters: ?1, ?2 correspond to method parameters.
 - Implementation automatically handled by the framework.

VI. Database Table Naming

- Default table name derived from entity class name (e.g., Passenger → passenger).
- Customization possible using JPA annotations.

I. Customized Method Implementation**A. @Query Annotation with JPQL/SQL (Second Approach)**

- Allows defining fully custom queries with maximum flexibility.
- **Mechanism:** Method name can be arbitrary; @Query annotation holds the JPQL or SQL query.
- **Example:** Retrieve passengers by first or last name.
- **Usage:** Defined in repository interface, invoked from runner or service class.
- **Real-World Application:** Useful for entities with many properties (e.g., 50+ fields).
- **Recommendation:** Preferred approach when standard JPA repository methods are insufficient.

B. @NamedQuery at Entity Level (Third Approach)

- Centralizes queries within the entity class for readability and reuse.
- **Mechanism:** @NamedQuery defines a unique name and JPQL/SQL query.
- **Example Query:** Retrieve passengers with seat number greater than a specified value.
 - name="passenger.getPassengersBySeat"
 - query="select P from passenger P where P.seatNumber > :seatNumber"
- **Parameter Handling:** Use @Param if method parameter name differs from named query parameter to avoid exceptions.
- **Benefit:** Predefined, reusable queries tied directly to the entity.

II. Paging and Sorting Concepts**A. Sorting**

- Allows displaying data in a specific order (ascending or descending).
- **Mechanism:** Uses Spring Data Sort class.
- **Ascending Order:** Sort.by(firstName)
- **Descending Order:** Sort.by(firstName).descending()
- **Multiple Fields:** Chain with and():
 - Example: Sort.by(firstName).and(Sort.by(seatNumber).descending())
- **Usage:** Pass Sort object to repository method: repository.findAll(sort)

B. Pagination

- Displays data in pages to improve performance and usability.
- **Key Interface:** Pageable
- **Implementation:** PageRequest
- **Constructing Pageable Object:**
- Pageable pageable = PageRequest.of(pageIndex, recordsPerPage, optionalSortObject);
 - pageIndex: zero-based page number
 - recordsPerPage: number of records per page
 - optionalSortObject: Sort object for combined sorting
- **Retrieving Paged Data:** Pass Pageable to repository: repository.findAll(pageable) → returns Page object
- **Controller & Service Roles:**
 - Service: create Pageable and call findAll(pageable)
 - Controller: extract content (page.getContent()) and navigation info
- **Navigation Methods:**
 - page.hasNext(): Is there a next page?
 - page.hasPrevious(): Is there a previous page?
 - page.getNumber(): Current page number

This summary highlights **custom query strategies** and **efficient data handling with sorting and pagination**, essential for scalable Spring Boot applications.

1. Practical Implementation of Pagination

1.1 Project Setup and Dependencies

- Create a Spring Starter Project (e.g., "Pagination app").
- **Dependencies:**
 - Spring Data JPA → database interaction & repository creation
 - Spring Web → building web apps & HTTP handling
 - Jasper (org.apache.tomcat.embed:tomcat-embed-jasper) → JSP support
 - JSTL (jstl:jstl) → use JSTL tags in JSP pages
 - MySQL driver → database connectivity
- Note: Jasper and JSTL are external libraries, not part of Spring Boot.

1.2 Core Components

- **Entity:** Maps to database table (e.g., Employee), includes fields like emp_number, e_name, salary, dept_number.
- **Repository:** Extends JpaRepository (e.g., PaginationRepository) for CRUD and pagination.
- **Service:** Handles business logic.
 - Core pagination: Pageable pageable = PageRequest.of(pageIndex, recordsPerPage)
 - Controls which data is displayed and in what order.
- **Controller:** Handles HTTP requests (e.g., /pagination)
 - Uses @RequestParam for page index (default = 0)
 - Stores data in ModelMap
 - Retrieves paginated data via service: page.getContent()
 - Provides navigation info: page.hasNext(), page.hasPrevious()

1.3 Front-End (JSP) Implementation

- **JSTL Tags:** Use c:if, c:forEach, c:out for conditional rendering, iteration, and data display.
- **Data Display:** Iterate through employee list, display properties (e.g., emp_number).
- **Navigation Buttons:**
 - "Next" → increments currentPage by 1
 - "Back" → decrements currentPage by 1

- Visibility depends on `page.hasNext()` and `page.hasPrevious()`

1.4 Application Configuration

- **Properties in `application.properties`:**
 - `server.port` → e.g., 2121
 - `server.servlet.context-path` → e.g., `/pagination`
 - Database connection → `spring.datasource.*`
 - JPA properties → `spring.jpa.hibernate.ddl-auto=update`, `spring.jpa.show-sql=true`

1.5 Back-End vs. Front-End Responsibilities

- Back-end: Primary responsibility for fetching paginated data.
- Front-end: Display data & navigation controls (JSP, React, Angular, etc.).
- Back-end developers may need to handle UI in small teams or client requirements.

2. Introduction to Relations (Associations)

2.1 Purpose

- Organize data in a structured format for efficient management and retrieval.

2.2 Real-World Examples

- Passport registration → Country → State → District hierarchy
- Demart/Shopping centers → Cities linked to stores
- Customer purchase history → Customer ID linked to transactions/items
- Loan eligibility → Customer ID linked to loan records and income details

2.3 Types of Relations

- Speaker mentions **four main types** (details in subsequent session).

2.4 Pervasiveness

- Relations are fundamental in almost all projects.
- Present in technologies like Spring Data JPA, IBM, TopLink, JPA, IBS, etc.
- Implementation details vary across platforms.

3. Developer Responsibilities and Soft Skills

- **Primary & Secondary Skills:**

SPRING BOOT & MICRO SERVICES

- Primary: Java, Spring Boot, Microservices
- Secondary: JSP, PHP, Angular, Oracle, MySQL
- **Client Expectations:** Developers often expected to be “end-to-end” capable.
- **Managing Expectations:** Demonstrate willingness to learn, e.g., "I can do this but need some time."
- **Adaptability:** Learn new tools as needed (Git, Bamboo/Jenkins, Linux/Unix).

Main Themes

- Focus on **practical application of relational database concepts**.
- Emphasizes **organizing data using relations** for efficient retrieval and management.
- Provides **step-by-step implementation** of the **one-to-many** relationship in a development context.

1. Purpose of Relations

- Relations (associations) organize data in an **organizational format**.
- Ensure **structured linkage** between different entities for efficient data management.

2. Four Types of Relationships

1. **One-to-Many**
 2. **Many-to-One**
 3. **Many-to-Many**
 4. **One-to-One**
- Most commonly used: **One-to-Many, Many-to-Many, One-to-One**.

3. One-to-Many Relationship Explained

- **Definition:**
 - One entity is linked to multiple related entities. Examples:
 - One customer → many items
 - One country → many states
 - One customer → many loans (gold, education, housing, personal)
- **Requirement-Driven:** Only implement a relationship if **business requirements** dictate.

4. Rules for Implementing One-to-Many

1. **Define Two Entities:** Parent entity and Child entity.
2. **Use @OneToMany Annotation:** Marks the parent-child relationship.
3. **Primary Key & Foreign Key Concept:** Maintain relationship integrity.

4. **@JoinColumn Annotation:** Specifies the foreign key column linking child to parent.
5. **Child as Property of Parent:** Include the child entity as a collection property inside the parent entity.

5. Practical Implementation Example (Customer and Loan)

Child Entity: Loan

- Annotated with @Entity and @Id.
- Fields: loanID (PK), loanType, amount.
- Example:

@Entity

```
public class Loan {
```

```
    @Id
```

```
    private String loanID;
```

```
    private String loanType;
```

```
    private double amount;
```

```
    // Getters and Setters
```

```
}
```

Parent Entity: Customer

- Annotated with @Entity and @Id.
- Fields: customerID (PK), customerName.
- Includes **Set<Loan>** to represent multiple child objects.
- Uses **@OneToMany(cascade = CascadeType.ALL)** → cascades save/update/delete operations.
- Uses **@JoinColumn(name = "customerID_fk")** → defines FK in Loan table.
- Example:

@Entity

```
public class Customer {
```

```
    @Id
```

```
    private int customerID;
```

```
    private String customerName;
```

```
@OneToMany(cascade = CascadeType.ALL)
```

```
@JoinColumn(name = "customerID_fk")
```

```
private Set<Loan> loans;
```

```
// Getters and Setters
```

```
}
```

Database Table Structure

- **loan table:** loanID (PK), loanType, amount, customerID_fk (FK)
- **customer table:** customerID (PK), customerName
- Relationship: customerID_fk references customerID

6. Development Workflow Steps

1. **Project Setup:** Create Spring Starter Project with Spring Data JPA & MySQL dependencies.
2. **Entity Creation:** Define Customer and Loan entities with proper annotations and fields.
3. **Repository Creation:**
 - LoanRepository and CustomerRepository extend JpaRepository for CRUD operations.
4. **Client Application (Runner):**
 - Use @Component and @Autowired to inject CustomerRepository.
 - Create Customer and Loan objects.
 - Associate multiple Loan objects with a single Customer using a collection (e.g., Set).
 - **Cascading Effect:** Saving the Customer automatically saves all associated Loan objects.

Summary

- The one-to-many relationship is **essential for real-world applications** where one entity has multiple related entities.
- Key implementation points:
 - Use **@OneToMany**, **@JoinColumn**, and **CascadeType.ALL** appropriately.
 - Maintain **primary and foreign key integrity**.
 - Implementation is **requirement-driven**.
- Demonstrates how saving a **parent entity automatically manages child entities**, ensuring database consistency and simplifying development.

1. One-to-Many Relationship Setup

- One **Customer** can have multiple **Loan** objects.
- Implemented by creating a **Collection (HashSet)** of Loan objects inside the Customer entity.
- **Cascade Property** ensures that saving the Customer also saves all associated Loan objects.
- **Quote:**

"I'm saving here customer but loans also saved into the database... because I use cascade = CascadeType.ALL..."

2. Database Schema Generation and Management

- **application.properties** controls database table behavior:
 - create: Generates new tables, dropping conflicting existing ones.
 - update: Uses existing tables, preserving data.
- Primary and foreign key constraints are automatically handled.
- **Quotes:**

"If you're using update maybe if it is existing already the database tables then existing database tables are utilized..."

"Now we can observe drop table if existing customer drop table if existing loan then create customer create loan then next alter primary key and foreign key..."

3. Lazy vs. Eager Loading

- **Lazy Loading (Default):**
 - Customer objects are loaded first; associated Loan objects are loaded only when accessed.
 - **Quotes:**

"by default one to many relationship is lazy loading"

"Lazy loading means first all the customers are going to be loaded. Whenever we are trying to iterating the first customer at the time loans are going to be loaded."

- **Eager Loading:**
 - Loads Customer and all associated Loans immediately.
 - Set via fetch = FetchType.EAGER in @OneToMany.
 - **Quote:**

"Early loading means when we are trying to loading these customers along with the customers all The loans also going to be loaded."

4. Object States (Transient, Persistent, Detached)

- **Transient:** Object created with new but not associated with a database session.
 - **Quote:** "This is the transient state."
- **Persistent:** Object is managed by persistence context; changes auto-reflected in DB.
 - **Quote:** "Modifications on persistent object automatically affect the database."
- **Detached:** Object was persistent but session is closed; changes not auto-synchronized.
 - **Quote:** "Delete by ID then 1001. This is what state detached state."

5. Adding a Loan to Existing Loans

- Load Customer into **persistent state**.
- Create new Loan (transient) and add to Customer's loan collection.
- **Transactional Requirement:**
 - Use @Transactional to ensure **atomic operations** (insert Loan + update FK).
 - **Quote:** "If two or more operations wants to perform, we have to use here one annotation that is at the rate of transactional."

6. Deleting a Loan

- **Removing Relationship Only (Orphan Records):**
 - Removing Loan from collection without orphanRemoval deletes only the association; Loan record may remain.
 - **Quote:** "It is removed only relationship... Now this type of records we call them as orphan records."
- **Deleting Complete Record:**
 - Use orphanRemoval = true in @OneToMany annotation.
 - Ensures the Loan entity is fully deleted from the database.
 - **Quote:** "We need to use here orphanRemoval = true. So that complete record is going to be deleted."

Most Important Ideas / Facts

- cascade = CascadeType.ALL automatically persists associated entities.
- **Default FetchType.LAZY** optimizes performance; use EAGER when needed.
- Understanding **object states (Transient, Persistent, Detached)** is essential for persistence management.
- @Transactional ensures atomicity when multiple database operations are involved.
- orphanRemoval = true prevents orphan records by deleting child entities when dissociated.
- save() acts as **insert or update** depending on entity state.

I. Many-to-One Relationship**Core Concepts & Characteristics**

- Opposite of One-to-Many.
 - **Quote:** "The many to one relationship is exactly opposite to one to many relationship."
- **Child-to-Parent Operations:** Saving a child automatically saves its parent; retrieving a child loads the parent.
 - **Quotes:**
 - "When I'm trying to saving the child object then automatically parent object also saved into the database."
 - "When we are trying So retrieving the child object then automatically parent object also came."
- **Default Loading:** Eager loading (FetchType.EAGER) by default; can override with fetch = FetchType.LAZY.
- **Foreign Key:** Defined using @JoinColumn linking child to parent (e.g., customer_id_fk).

Implementation Rules

1. Define two entities: **Parent** and **Child**.
2. Parent entity must be defined as a **property in the Child entity**.
3. Use @ManyToOne annotation on the parent property in the child entity.

Practical Example (Customer and Loan)

- Customer = Parent, Loan = Child (many loans to one customer).
- Loan entity includes loanId, loanType, amount, and a Customer customer property.
- Annotations: @ManyToOne and @JoinColumn(name = "customer_id_fk").
- Cascade operations using CascadeType.ALL ensure parent is saved/updated along with child.

Saving Mechanism

- Set Customer on each Loan object:
 - personal.setCustomer(customer);
 - home.setCustomer(customer);
- Save Loan objects using repository:
 - loanRepo.save(personal);

- `loanRepo.save(home);`
- Customer object is automatically saved or updated due to cascading.

II. Many-to-Many Relationship

Core Concepts & Characteristics

- Reverse of One-to-Many/Many-to-One.
- Use cases include:
 - **Student-Course:** A student can enroll in many courses; a course can have many students.
 - **Customer-Loan (alternate perspective):** A loan type can be taken by many customers.
 - **Customer-Item:** Customers can buy multiple items; items can be bought by multiple customers.

Implementation Rules

1. Define two entities: parent and child.
2. Use `@ManyToMany` annotation on the relationship.
3. Maintain a **join table** to store foreign key references between the two entities.
4. Use `@JoinTable` annotation to define the join table and its columns.

Practical Example (Customer and Item)

- **Entities:**
 - Customer: `customer_ID`, `customer_name`
 - Item: `item_ID`, `item_name`, `price`
- **Join Table:** `customer_items`
 - Columns: `customer_ID_FK` (FK → Customer), `item_ID_FK` (FK → Item)
- Stores many-to-many relationships: a customer can have multiple items, and an item can belong to multiple customers.

III. General Implementation Notes & Best Practices

- **Relationship Selection:** Always based on **business requirements**.
- **Project Setup:** Spring Starter Project with Spring Data JPA.
- **Database Interaction:** `application.properties` for connection and schema management (create for initial table creation).

- **Component & Dependency Injection:**
 - @Component for marking classes as Spring components (e.g., runner).
 - @Autowired to inject repositories (e.g., LoanRepository).
- **Practice:** Hands-on implementation is critical to fully understand relationships.
 - **Quote:** "If you want to get complete check then you have to practice yourself also."

I. Many-to-One Relationship**Definition:**

- Represents multiple child entities associated with a single parent entity.
- Opposite of One-to-Many: Saving a child automatically saves the parent.
 - **Quote:** "The many to one relationship is exactly opposite to one to many relationship."

Key Characteristics & Rules:

- Operations are **Child-to-Parent**: saving/retrieving a child also affects the parent.
- Requires **two entities**: Parent and Child.
- Parent entity is defined as a **property inside the child entity**.
- Declared using **@ManyToOne** annotation.
- Default Loading: **Eager (early loading)**, can override with `fetch = FetchType.LAZY`.
- Foreign Key defined using **@JoinColumn(name = "customer_id_fk")** linking child to parent.

Implementation Example (Customer and Loan):

- **Customer (Parent)**: `customerID`, `customerName`.
- **Loan (Child)**: `loanID`, `loanType`, `amount`.
- Loan entity includes a Customer property:
- `@ManyToOne(cascade = CascadeType.ALL)`
- `@JoinColumn(name = "customer_id_fk")`
- `private Customer customer;`
- Cascade ensures saving Loan also saves/updates Customer.

Operational Flow:

1. Create entities: Customer and Loan.
2. Define repositories: `CustomerRepository` and `LoanRepository`.
3. Instantiate objects in client application (e.g., `MyRunner`).
4. Set Customer on each Loan object:
5. `personal.setCustomer(customer);`
6. `home.setCustomer(customer);`
7. Save Loan objects using repository: `loanRepo.save(personal)` and `loanRepo.save(home)`.

8. Database tables created: customer and loan, with customer_id_fk in Loan table.
9. Retrieving a child object also loads its parent automatically.

II. Many-to-Many Relationship

Definition:

- Multiple instances of one entity associated with multiple instances of another entity.
- Requires **specific business requirements** to determine usage.

Examples:

- Students ↔ Courses: A student can take multiple courses; a course can have multiple students.
- Customers ↔ Loans: A loan type can be associated with multiple customers.
- Customers ↔ Items: Customers can buy multiple items; items can be purchased by multiple customers.

Key Characteristics & Rules:

- Requires **two entities**: parent and child.
- Declared using **@ManyToMany** annotation.
- Requires a **join table** to resolve the many-to-many relationship.
 - Join table stores foreign key references of both entities.
 - Specified using **@JoinTable** annotation.

Conceptual Example (Customer and Item):

- **Customer Table**: customer_ID (PK), customer_name.
- **Item Table**: item_ID (PK), item_name, price.
- **Join Table (customer_items)**:
 - customer_ID_FK → Customer
 - item_ID_FK → Item
- Join table manages the relationships between customers and items.

Note: Actual Many-to-Many implementation details are deferred to a subsequent session.

III. General Best Practices & Considerations

- **Practice is essential:** Hands-on implementation is required to fully understand concepts.

SPRING BOOT & MICRO SERVICES

- **Quote:** "You have to practice yourself also."
- **BRD Importance:** A clear Business Requirement Document ensures proper relationship implementation.
 - **Quote:** "If the BRD document is clear then nothing is there implementation business requirement document."
- **Common Relationships in Applications:** One-to-Many, Many-to-Many, and One-to-One are most frequently used.

1. Many-to-Many Relationship Implementation

Definition & Concept:

- Multiple instances of one entity associated with multiple instances of another.
- Example: One Customer can have many Items, and one Item can belong to many Customers.
- Implemented in Spring Data JPA using **@ManyToMany** and a **join table**.

Implementation Steps:

- **Entities:** Customer and Item defined using **@Entity**.
- **Primary Keys:** Each entity has **@Id** (e.g., customerId, itemId).
- **Customer Entity:**
 - Field: private List<Item> items;
 - **@ManyToMany(cascade = CascadeType.ALL)** for relationship and cascading operations.
 - **@JoinTable(name = "customer_items", joinColumns = @JoinColumn(name = "customerId_fk"), inverseJoinColumns = @JoinColumn(name = "itemId_fk"))** defines the join table.
- **Item Entity:** Can have similar mapping if bi-directional relationship is needed.
- **Repositories:** CustomerRepository and ItemRepository extend **JpaRepository**.
- **Runner Class (MyRunner):**
 - Instantiates multiple Customer and Item objects.
 - Associates Item lists with Customer objects using `c1.setItems(itemListOne);`.
 - Saves Customers: `customerRepository.save(c1); customerRepository.save(c2);`
 - Cascade ensures Item objects and join table entries are persisted automatically.
- **Verification:** Observe SQL queries and database tables (customer, item, customer_items).

Key Quotes:

- "Join table contains the foreign key values on it."
- "1 customer is taken by the many items... one item is going to be taken by the many customer."

2. One-to-One Relationship

Definition & Concept:

- One instance of an entity is associated with exactly one instance of another entity.
- Operations can be performed **parent-to-child** or **child-to-parent**.
- Suitable for unique mappings only (e.g., Customer → PAN card, Employee → Passport).

Implementation Rules:

- Define **two entities**: parent and child.
- Annotate relationship with **@OneToOne**.
- Foreign key should **not allow duplicate values**.

Key Quote:

- "Even though foreign key is there should not allow duplicate values."

3. Data Binding and JPA (Brief Discussion)**Concept:**

- Data binding involves connecting a data model (e.g., from a form) to entity objects.
- Example discussed: JSP/JDBC pagination app.
- Spring Data JPA integration requires additional learning and tutorials.

Key Quote:

- "CRM application we are binding some data model... we have to look into some tutorials... try to figure out."

4. Overall Takeaways

- JPA annotations like **@ManyToMany**, **@OneToOne**, **@JoinTable**, **@JoinColumn**, **cascade** manage complex entity relationships.
- Relationship type selection must align with **business requirements**.
- Practical implementation steps: define entities → configure annotations → use repositories → test with a runner class.
- Always **verify database schema and persisted data** to confirm relationship mapping.
- Data binding with JPA is more complex than JDBC and requires additional exploration.

I. One-to-One Relationship Implementation**A. Entity Definition**

- **Person Entity**
 - Annotated with @Entity.
 - Primary key: personID (@Id).
 - Field: personName (optionally @Column(length = 15) to reduce column size).
 - Getters and setters included.
- **Passport Entity**
 - Annotated with @Entity.
 - Primary key: passportNumber (@Id).
 - Field: expireDate of type LocalDate (Java 1.8 feature).
 - Getters and setters included.

B. Defining the One-to-One Relationship

- Declared within **Passport entity**:
 - @OneToOne(cascade = CascadeType.ALL) ensures all operations (save, retrieve, delete) cascade from Passport to Person.
 - @JoinColumn(name = "personID_FK", unique = true) defines the foreign key column and enforces uniqueness.
 - Example field: private Person person;
- **Cascade Effects:**
 - Saving a Passport also saves the associated Person.
 - Retrieving a Passport automatically retrieves the linked Person.

C. Repository Layer

- Interfaces marked with @Repository.
- Extend JpaRepository<Entity, ID_Type> for CRUD operations.
- Examples:

- PassportRepository extends JpaRepository<Passport, Integer>
- PersonRepository extends JpaRepository<Person, Integer>

D. Client Application (Testing)

- @Component marks the testing class.
- @Autowired injects PassportRepository.
- Create **Person** and **Passport** objects, set the Person in Passport, and save:
 - passportRepo.save(passport);
 - Cascade ensures both entities are persisted simultaneously.
- Expire date can be set using LocalDate.of(year, month, day).

E. Database Configuration

- application.properties
 - spring.jpa.hibernate.ddl-auto=create creates tables from scratch at application startup.

II. Introduction to Database Joins**A. Purpose of Joins**

- Fetch data from **multiple tables**.
- Joins are **read-only**: cannot perform delete or update operations.

B. Types of Joins

1. **Inner Join** – returns records where keys match in both tables.
 2. **Left Outer Join** – returns all records from the left table, nulls for unmatched right-side rows.
 3. **Right Outer Join** – returns all records from the right table, nulls for unmatched left-side rows.
 4. **Full Outer Join** – combination of left and right outer joins (less commonly used).
- Mostly used joins: **Inner Join, Left Outer Join, Right Outer Join**.

C. How Inner Join Works

- Matches **left table primary key** with **right table foreign key**.

- Example:
 - Customer table: customer_ID, customer_name
 - Loan table: loan_ID, loan_type, amount, customer_ID_FK
 - Only records where customer.customer_ID = loan.customer_ID_FK are returned.

D. Constructing Join Queries

1. **SQL Query** – uses actual table and column names:

```
SELECT customer_ID, customer_name, loan_type, amount
```

```
FROM Customer
```

```
JOIN Loan ON Customer.customer_ID = Loan.customer_ID_FK;
```

2. **JPQL Query** – object-oriented, uses entity classes and properties:

```
SELECT C.customerID, C.customerName, L.loanType, L.amount
```

```
FROM Customer C
```

```
JOIN C.loans L; // 'loans' is the collection property in Customer
```

- Note: Use **capital letters** for entity class names in JPQL.

E. Other Join Types (Overview)

- **Left Outer Join:** Returns all left-side records; nulls for unmatched right-side.
- **Right Outer Join:** Returns all right-side records; nulls for unmatched left-side.
- **Full Outer Join:** Combination of left and right joins.

Key Takeaways

- One-to-One relationships require **@OneToOne**, **@JoinColumn**, **cascade** for proper persistence.
- Cascade ensures synchronized save/retrieve between linked entities.
- Foreign key uniqueness maintains the integrity of one-to-one mappings.
- Database joins are essential for **retrieving related data** from multiple tables without modifying them.
- JPQL allows join queries at the **entity level**, aligning with object-oriented programming practices.

I. Database Joins: Practical Implementation**A. Inner Join**

- Returns only rows with a match in both tables based on the join condition.
- Example SQL:

```
SELECT C.customer_ID, C.customer_name, L.loan_ID, L.amount
```

```
FROM customer C
```

```
JOIN loan L ON C.customer_ID = L.customer_ID_FK;
```

- nativeQuery = true allows execution of raw SQL instead of JPQL.
- Key takeaway: Customers without matching loans (e.g., customer_ID 1002) are **not displayed**.

B. Left Outer Join

- Returns all rows from the **left table** and matching rows from the right table.
- If no match exists in the right table, **NULL values** appear for right-side columns.
- Key takeaway: Customers without loans are displayed with NULL for loan-related fields.

C. Right Outer Join

- Returns all rows from the **right table** and matching rows from the left table.
- If no match exists in the left table, **NULL values** appear for left-side columns.
- Key takeaway: All loan records are displayed; unmatched customers show NULL for left-side columns.

D. Full Outer Join (Mentioned)

- Returns all rows when there is a match in either table.
- Combination of left and right joins; less commonly used in practice.
- Key takeaway: Mostly theoretical or interview-focused.

II. Application Security: Fundamental Concepts

A. Authentication vs. Authorization

- **Authentication:** Verifies user identity (login).
 - Limitation: User may access the application but cannot perform operations.
 - Example: Logging into net banking without ability to transfer money.
- **Authorization:** Determines what actions an authenticated user is allowed to perform.
 - Example: After login, transferring money is allowed if authorized.
 - Real-world analogy: Bank employee may authenticate to process a check but requires manager authorization to complete a high-value transaction.

B. Types of Authentication

1. **In-Memory Authentication**
 - Credentials stored at application level.
 - Suitable for **small-scale applications**.
2. **JDBC Authentication**
 - Credentials stored at database level.
 - Recommended for **large-scale applications**.

C. Web Application Development for Security Implementation

- **Project Setup:** Spring Starter Project (e.g., Employee Management System).
- **Dependencies:** Spring Data JPA, JSP, MySQL.
- **Project Structure:**
 - com.ems.entity: Entity mapping (e.g., Emp with properties).
 - com.ems.repository: Data access layer (e.g., EmployeeRepository).
 - com.ems.model: DTOs for data transfer between layers.

III. Remaining Sessions and Practice

- Approximately **10 sessions** remaining; expected completion by **second week of June**.
- One-week break planned.
- Emphasis on **hands-on practice** to solidify understanding of joins and security concepts.

I. Database Procedures**A. Purpose and Advantages**

- **Improve Application Performance:**
 - Procedures execute complex logic on the backend, reducing time-consuming full-database queries.
 - Example: Direct queries “hit the total database,” while procedures fetch only required data.
- **Reduce Java Developer Burden:**
 - Encapsulate complex business logic in the database, simplifying Java code.
 - Less code, easier maintenance, and better scalability for multiple users.

B. Problem Statement & Solution

- **Issues with Direct SQL Queries:**
 - Hits entire database even for small requirements.
 - Time-consuming and inefficient.
 - Scalability problems when multiple users request data simultaneously.
- **Solution:**
 - Design a **backend procedure** to handle logic efficiently.
 - Application calls procedure, improving performance and reducing overhead.

C. Key Rules for Procedure Development

- **Database Table Required:** Procedures operate on tables.
- **Parameters:**
 - **IN:** For input values (e.g., empNumber).
 - **OUT:** For output values (e.g., employee name, salary).
 - **IN-OUT:** Rare cases; used for input and output simultaneously.

D. Practical Implementation Example: calculate_bonus

- **Syntax:**

CREATE OR REPLACE PROCEDURE calculate_bonus (

in_emp_num INT,

out_name VARCHAR(20),

out_bonus DOUBLE

);

- **Logic:**
 - Retrieve employee details.
 - Apply conditional bonus logic:
 - IF salary ≤ 20,000 → bonus = 20%
 - ELSE → bonus = 30%
- **Compilation:** Use delimiter (\$\$) for PL/SQL execution.

E. Calling Procedures from Spring Boot

1. Entity Class with @NamedStoredProcedureQuery:

- Maps procedure parameters (IN/OUT).
- Example:
 - name = "readBonus"
 - procedureName = "calculate_bonus"

2. Repository Interface with @Procedure:

- Custom method annotated to call procedure.
- Extends JpaRepository.

3. Runner Class:

- Auto-wires repository.
- Calls procedure method with input and receives output.
- Enables Java application to leverage backend procedure efficiency.

II. Security

A. Importance of Security

- Mandatory for all web applications.

- Without security: "Everyone can access and perform operations → application becomes useless."

B. Key Terminology

- **Authentication:**
 - Verifies identity; login only.
 - Example: Viewing bank balance without permission to modify it.
- **Authorization:**
 - Determines actions allowed for authenticated users.
 - Example: Transferring money between accounts after login requires authorization.

C. Types of Authentication

1. **In-Memory Authentication:**
 - Credentials configured at application level.
 - Suitable for **small-scale applications**.
2. **JDBC Authentication:**
 - Credentials stored in and verified against the database.
 - Recommended for **large-scale applications** (90% of cases).

D. Future Steps

- Upcoming session: Build a web application and implement security features.
- Focus: Practical application of authentication and authorization concepts.

1. Web Application Development Fundamentals

- Basic Spring Boot web application structure:
 - Requests map to JSP pages (e.g., index.jsp, addEmployee.jsp).
 - Controllers handle actions (addEmployee, saveEmployee).
 - Data binding via @ModelAttribute.
 - Persistence using Spring Data JPA or JDBC.
 - application.properties configures server port, context path, database connection, and JPA properties.
- Core functionalities: Adding, editing, deleting employee records.

2. Security Vulnerabilities in Basic Web Apps

- Without explicit security:
 - "Everyone can open this website and perform operations."
 - No access control or authentication is present.

3. Introduction to Spring Security

- Framework for securing Spring Boot applications.
- Provides authentication, authorization, and access control mechanisms.

4. In-Memory Authentication

- User credentials and roles are **hardcoded** in the application configuration.
- Configuration steps:
 - Add dependency: spring-boot-starter-security in pom.xml.
 - Create a configuration class annotated with @Configuration and @EnableWebSecurity.
 - Extend WebSecurityConfigurerAdapter to override security methods.
- Example user configuration (inside configureGlobal method):
 - dura → password dura123, role manager
 - abinos → password abinos123, role user
- Passwords are encoded with BCryptPasswordEncoder for secure storage.

5. Role-Based Authorization

- Access control via roles:
 - Example: /delete** URL accessible **only** by manager.
 - Any other URL accessible to all users.
 - Unauthorized users redirected to custom error page (deny.jsp).
 - Custom error pages improve clarity compared to generic white-label errors.
-

6. Error Handling and User Experience

- `http.exceptionHandling().accessDeniedPage("/deny.jsp")` used for unauthorized access.
 - User-friendly messages communicate access issues clearly.
 - Prevents confusion caused by default Spring Security error pages.
-

7. Limitations of In-Memory Authentication

- **Low Security:** Credentials are hardcoded and visible in code.
 - **Scalability:** Suitable only for small-scale applications (1–2 users).
 - **Not Suitable for Enterprise Applications:** Cannot handle large numbers of users or dynamic credential management.
 - **Cost Implication:** Reduces cost for small projects by avoiding a database for authentication.
-

8. Transition to JDBC Authentication (Future Topic)

- JDBC authentication addresses limitations:
 - Dynamic user management.
 - Improved security for large-scale applications.
 - Integration with a database for credential storage.
-

Key Takeaways

- Progression: unsecured web app → secured with basic in-memory authentication.
 - Role-based access control ensures only authorized users can perform sensitive actions.
 - In-memory authentication is **good for demonstration and small apps**, but **not recommended for production or large-scale systems**.
 - Future implementation with JDBC authentication is necessary for enterprise-grade security.
-

1. JDBC Authentication vs. In-Memory Authentication

- **Problem with In-Memory Authentication**
 - Credentials hardcoded in config files.
 - Passwords visible if files are accessed.
 - Unsuitable for large-scale apps.
- **Advantages of JDBC Authentication**
 - Credentials stored in **encrypted format** inside database.
 - Even with DB access, passwords remain unreadable.
 - Recommended for **enterprise / large-scale applications**.

2. Implementing JDBC Authentication

- **Database Schema Requirements**
 - **users table:**
 - username (PK, VARCHAR(20))
 - password (VARCHAR(150), encrypted with BCrypt)
 - enabled (boolean/int → 1 = active, 0 = disabled)
 - **authorities table:**
 - username (FK → users.username)
 - authority (role name, VARCHAR(20))
- **Special Considerations**
 - Encrypted passwords require larger column size.
 - Accounts can be disabled without deletion (e.g., employee retirement).
- **Dependencies & Setup**
 - Add spring-boot-starter-data-jdbc in pom.xml.
 - Use **BCryptPasswordEncoder** for password hashing.
 - Insert initial user/role data with CommandLineRunner.
- **Security Configuration**
 - Replace inMemoryAuthentication with jdbcAuthentication.

- Provide a DataSource bean.
- Define custom SQL queries:
 - usersByUsernameQuery → fetches user credentials.
 - authoritiesByUsernameQuery → fetches user roles.
- Configure passwordEncoder for authentication.

3. Application Security Enhancements

- **Securing Index Page**
 - Change from permitAll() → authenticated().
 - Index accessible only after login.
- **Form-Based Login**
 - Replaces HTTP Basic auth.
 - Provides a user-friendly login page.
- **Authentication vs. Authorization**
 - **Authentication:** Verifies identity (username/password).
 - **Authorization:** Defines what actions a user can perform.
 - Example:
 - *avinash* = authenticated but unauthorized to delete.
 - *dura* = authenticated + authorized to delete.
- **Session Management**
 - **Maximum Sessions:** Restrict to 1 session per user.
 - Prevents multiple concurrent logins (important in banking apps).
 - Example: logging in from another browser expires first session.
 - **Session Timeout:** Auto logout after inactivity.
 - Config via server.servlet.session.timeout=2m.
 - Analogy: ATM auto logout after 2–3 minutes.

4. Development Workflow Improvements (DevTools)

- **Problem:** Manual server restart required after each code change.

- **Solution:** Add spring-boot-devtools dependency.
- **Benefit:**
 - Automatic recompilation.
 - Automatic rebuild and redeployment.
 - No need to restart server manually for small changes.

5. General Development Principles

- **Client Focus:** Implement requirements quickly to meet client expectations.
 - **Rapid Implementation:** Speed is essential in real projects.
 - **Beyond Development:** Projects require testing, deployment, and continuous updates.
 - **Self-Reliance:** Developers must upskill quickly (e.g., within a week).
-

1. Need for Web Services & Middleware

- Web services solve communication between apps built in different languages (Java, .NET, PHP, Angular, etc.).
- Middleware technology (web services) ensures interoperability.
- Example: Java backend ↔ .NET frontend interaction.

2. Evolution of Web Services: SOAP vs REST

- **SOAP (Simple Object Access Protocol):**
 - XML-based, verbose, "heavyweight."
 - Relies on WSDL files.
 - Rolled out of use due to complexity.
- **REST (Representational State Transfer):**
 - Lightweight, annotation-based.
 - No XML file generation—just Java code.
 - Uses JSON as the preferred data format.
 - Simpler, faster, and widely adopted in modern apps.

3. Advantages of Spring Boot for REST

- Traditional services required deployment to external servers (Tomcat, WebLogic, JBoss).
- Spring Boot eliminates this dependency by embedding a server.
- Simplifies REST service deployment and boosts adoption.

4. REST: Representation, State, Transfer

- **Representation:** Data format (XML, JSON, or plain text).
 - XML example: `<EMP><EMP_number>101</EMP_number></EMP>`
 - JSON example: `{"emp_number": "101", "ename": "Scott"}`
 - JSON is most common today (lightweight, human-readable).
- **State:** Actual data (e.g., employee details).

- **Transfer:** Moving data from one layer to another.

5. Key Annotations in Spring Boot REST

- **@RestController:** Combines @Controller + @ResponseBody. Defines REST endpoints.
- **@RequestMapping / @GetMapping / @PostMapping / @PutMapping / @DeleteMapping:** Map HTTP methods to handler functions.
- **@RequestBody:** Converts incoming JSON into Java entity objects. Required before storing in DB.
- **@ResponseBody:** Converts Java objects into JSON/XML responses.

6. Accepting Client Input

- **Path Variables (@PathVariable):**
 - Values embedded in URL.
 - Example: /search/{source}/{destination} → /search/Hyderabad/Delhi.
- **Query Parameters (@RequestParam):**
 - Values sent as key-value pairs after ?.
 - Example: /search?source=Hyderabad&destination=Delhi.

7. Spring Boot Project Setup

- Use **Spring Initializr** to generate project.
- Example project name: com.pms (Project Management System).
- Required dependencies:
 - **Spring Web** (for REST APIs).
 - **Spring Data JPA** (for DB interaction).
- Versions: Use stable JDK (e.g., 1.8) and compatible Spring Boot version.
- Post-setup: Right-click → "Update project."

✓ Key Takeaway:

Spring Boot makes developing RESTful web services **simple, fast, and server-independent**. REST (with JSON) replaces SOAP as the dominant approach, and annotations like @RestController, @RequestBody, and @PathVariable streamline API development.

Spring Boot REST API Development and Swagger Implementation

1. REST API Architecture in Spring Boot

- Follows a layered structure: **Entity** → **Repository** → **Service** → **DTO (Model)** → **Controller**.
- **Entity**: Maps Java classes to database tables/columns.
- **Repository**: Extends JpaRepository for database persistence & custom queries (e.g., findByManufacturer).
- **Service/ServiceImpl**: Contains business logic (CRUD operations like search, add, update, delete).
- **DTO (Model)**: Serializable objects for transferring data across layers/networks.
- **Controller (@RestController)**: Receives client requests, maps them to service methods.

2. Core CRUD Operations

- **Create** → Insert new records.
- **Read** → Fetch product(s) by ID, manufacturer, or all.
- **Update** → Modify existing product data.
- **Delete** → Remove product by ID.

3. REST Communication (Consumes vs Produces)

- **Produces (application/json)**: API → Browser (e.g., GET request sending JSON).
- **Consumes (application/json)**: Browser → API (e.g., POST/PUT request sending JSON to insert/update data).

4. Database Integration

- Connected to **Oracle Database** using dependencies (com.oracle, ojdbc6, version 7.2.0.4).
- Entity classes directly map to database tables.
- Queries like SELECT * FROM PRODUCTS validate data persistence.

5. Testing APIs: Browser, Postman, Swagger

- **Browser testing** is limited (only GET requests work easily).

- **Postman:** Developer tool for full CRUD testing with JSON payloads.
 - **Problem:** Not user-friendly for non-technical users.
-

6. Swagger for Documentation & Testing

- **Need:** Provides an easy browser-based interface for testing REST APIs.
 - **Dependencies:** springfox-swagger2, springfox-swagger-ui (v2.9.2 used).
 - **Setup Steps:**
 - Add Swagger dependencies.
 - Enable with `@EnableSwagger2` on main application class.
 - Create a **Docket Bean** to configure Swagger and scan base packages (com.pms.controller).
 - Use `@ApiOperation` on controller methods for meaningful endpoint descriptions.
 - **Swagger UI URL:** <http://localhost:8080/productAPI/swagger-ui.html>.
-

7. Benefits of Swagger

- **Interactive Documentation:** Auto-generated for all endpoints.
 - **Direct Testing in Browser:** Eliminates need for Postman for CRUD operations.
 - **User-Friendly:** Makes REST APIs accessible to non-developers ("ordinary customers").
 - **Supports Full CRUD:** GET, POST, PUT, DELETE all possible via browser UI.
-

Caching Techniques and Profiles in Spring Boot

I. Caching Technique

Main Theme: Improve application performance by reducing unnecessary database hits via cache memory.

- **Purpose of Caching**

- Boosts real-time performance by serving frequently accessed data from cache.
- Reduces database load and improves response time.

- **How Caching Works (Flow)**

1. First request → Application queries database → Data stored in cache + returned to user.
2. Subsequent request → Application checks cache first.
3. **Cache Hit** → Data returned from cache (DB not queried).
4. **Cache Miss** → Query DB → Store in cache → Return to user.

- **Performance Verification**

- If SQL queries appear → Data fetched from DB.
- If no SQL queries generated → Data served from cache.

- **Caching with Redis**

- Redis server = “Redis cache” for Spring Boot integration.
- Auto memory management: Unlike Hibernate’s manual cache capacity tuning, Redis handles increase/decrease automatically.

- **Spring Boot Caching Dependencies**

- spring-boot-starter-cache
- spring-boot-starter-data-redis

- **Redis Setup**

- Download and start Redis server.
- Verify via redis-cli → ping (response: pong).
- Check cache keys: keys *
- Clear cache: flushall

- **Integration with Spring Boot**

- application.properties:
 - spring.cache.type=redis
 - spring.redis.host=localhost
 - spring.redis.port=6379
- **Annotations:**
 - @EnableCaching → Enables caching at app level.
 - @Cacheable → Marks methods for caching.
 - value="allProductsCache" → Cache name.
 - key="#root.methodName" → Cache key.
- **Real-world Use Case**
 - High-traffic apps (e.g., IRCTC ticket booking) benefit from caching under heavy concurrent load.
 - Avoids hitting DB repeatedly for the same data.

II. Profiles Concept

Main Theme: Manage configurations for multiple environments (dev, test, UAT, prod) without touching application code.

- **Purpose of Profiles**
 - Environment-specific configurations (e.g., DB connections, logging).
 - Eliminates code changes when moving between environments.
- **How Profiles Work**
 - Create environment property files:
 - application-dev.properties
 - application-test.properties
 - application-uat.properties
 - Each contains environment-specific configurations.
- **Activation**
 - In application.properties:
 - spring.profiles.active=dev (or test/uat/prod).
 - On startup → Spring Boot loads configurations of the active profile automatically.

- **Example Scenarios**

- **Dev** → Oracle DB.
- **UAT** → MySQL DB.
- **CR** → DB2 DB.
- Switching profile changes DB source instantly without code modification.

- **Demonstration**

- Same Product ID queried in Oracle and MySQL returned different results depending on active profile.
- Confirms environment-specific configuration switching.

- **Decision-Making**

- Number of required environments (dev, test, sit, pre-prod, prod) decided by client/team lead.

✓ In short:

- **Caching** → Boosts performance by reducing DB hits (commonly implemented with Redis in Spring Boot).
- **Profiles** → Enable environment-specific configurations without modifying code, improving flexibility and maintainability.

I. Spring Boot Actuators

Main Theme: Provide monitoring, health checks, metrics, and internal application details—mainly for production teams.

- **Purpose**
 - Reveal internal state of a running application.
 - Help production teams monitor health, performance, and configuration.
- **Target Audience**
 - Primarily for **production/ops teams**.
 - Configured by developers.
- **Dependency**
 - Add spring-boot-starter-actuator in **pom.xml**.
- **Common Actuator Endpoints**
 - /actuator/health → Reports application health (e.g., "up and running", missing dependencies).
 - /actuator/info → Custom metadata (name, description, version).
 - /actuator/beans → Lists all Spring beans in the app context.
 - /actuator/mappings → Shows all REST endpoints (request mappings).
 - /actuator/metrics → JVM memory, garbage collection, threads (live thread count, etc.).
- **Enabling Endpoints**
 - By default, only /health and /info are exposed.
 - Must enable others in application.properties:
 - management.endpoints.web.exposure.include=*
 - management.endpoint.health.show-details=always
 - Custom info:
 - info.app.name=Spring Boot Rest Application
 - info.app.description=CRUD operations app
 - info.app.version=1.1

- **Usefulness**

- Production teams track memory usage, uptime, thread counts.
- Debugging + monitoring tool for live systems.

II. Introduction to Microservices

Main Theme: Break down monolithic applications into smaller, independently deployable services for better fault isolation and scalability.

- **Definition**

- "Monolithic application is divided into multiple smaller applications. Each application is one microservice."

- **Problems with Monolithic Applications**

- All functionality bundled in one app.
- **Lack of fault isolation:** One failed function (e.g., "transfer balance") can break the whole system (e.g., "withdraw" fails too).
- Scalability and maintenance challenges.

- **Microservices Approach**

- **One functionality → one microservice.**
- Independent deployment, testing, and scaling.
- Each microservice must handle **its own database** (database per service pattern).

- **Communication & Integration**

- Microservices interact with each other via REST APIs or messaging.
- Requires careful integration.

- **Example: Call Details Microservice**

- **Project Setup:** Spring Starter Project (micro-call-details).
- **Dependencies:** Spring Data JPA, Spring Web, MySQL connector.
- **Layers:**
 - Entity: CallDetails (fields: callId, callingNumber, receivingNumber, callDuration, callDate).
 - Repository: CallDetailsRepository (extends JpaRepository).
 - Service & Implementation: CallDetailsService, CallDetailsServiceImpl.

- DTO (Model): CallDetailsModel.
- Controller: CallDetailsController (REST API).
- **Database Config:** application.properties with dedicated DB (e.g., call_details_DB).
- **Functionality Example:** readCallDetails → retrieves call info by phone number.

Conclusion

- **Actuators:** Critical for monitoring, diagnostics, and production readiness.
- **Microservices:** Solve monolithic limitations by enabling modular, scalable, fault-tolerant systems.
- Practical example (Call Details service) illustrates how **Spring Boot** enables microservice development with isolated databases and RESTful endpoints.

I. Core Concepts of Microservices Development

- **Independent services in one project:** Multiple microservices can be developed within a single project, avoiding the misconception that each requires a separate project.
- **Dedicated databases per microservice:**
 - Each microservice must have its own database.
 - Different microservices can use different database technologies (e.g., Oracle, MySQL).
- **Standardized structure across microservices:**
 - **Entity package:** Defines database mappings (e.g., com.micro.plan.entity).
 - **Repository package:** Extends JPA repositories for persistence (e.g., com.micro.plan.repository).
 - **Service & implementation packages:** Business logic definition and implementation.
 - **Model package (optional):** DTOs or domain models.
 - **Controller package:** REST controllers exposing endpoints.
 - **application.properties:** Configures DB connections and application settings.

II. Practical Microservice Examples and Functionality

1. Micro Plan Details Service

- Maintains and exposes available plan information.
- Endpoints:
 - GET /plan/browse/plans → list all plans.
 - GET /plan/{id} → retrieve a specific plan.
- Connects to **plan DB**.
- Simple implementation: fetch data, iterate, copy properties, and return results.

2. Micro Call Details Service (previous session)

- Focused on call durations and related data.

3. Micro Friend Service

- Purpose: Retrieve friend contact numbers by phone number.
- Functions: Add friend, read friends, remove friend.
- Database: Separate DB for friend details.

- Key details:
 - Entity with auto primary key generation (GenerationType.AUTO).
 - Repository supports SQL/native queries (nativeQuery=true).
 - Service logic checks for existing contact before adding (returns 1 if exists, 0 otherwise).
 - Data persisted using repository.saveAndFlush().
- Example endpoint:
 - GET /friend/details/{phoneNumber} → fetch friend contacts.

III. Microservices Integration: RestTemplate

- **Need:** Services must integrate after independent development.
- **Tool:** Spring's RestTemplate enables inter-service communication.
- **Quote:** "In order to provide the communication between these two microservices, we have to use the predefined class called RestTemplate."
- **Example scenario:**
 - A **customer service** calls **plan** and **friend services** internally to fulfill requests.

IV. Developing a Customer Microservice (Next Steps)

- Follows the same layered pattern: entity, repository, service, controller.
- Acts as an **aggregator/orchestrator** of other microservices.
- **Controller** will demonstrate integration using RestTemplate.
- **UI Integration:** All microservices will eventually connect to a unified UI application.

V. Key Takeaways and Future Focus

- **Scalability & modularity:** Independent development, deployment, and scaling.
- **Database diversity:** Each service uses its own DB, possibly with different technologies.
- **Standardized patterns:** Entity → Repository → Service → Controller ensures maintainability.
- **RestTemplate backbone:** Essential for inter-microservice communication.
- **Future focus:**
 - Implement integration in the customer microservice controller.
 - Demonstrate integration of all services into a single UI application.

I. Microservices Architecture and Communication

- **Components of a Microservice**
 - **Models (DTOs):** Transfer data between layers/services (e.g., CustomerDTO, LoginDTO, PlanDTO).
 - **Service & Implementation:** Define and implement business logic.
 - **Controller (REST Controller):** Exposes functionality as REST endpoints.
- **Inter-service Communication with RestTemplate**
 - RestTemplate is used for making HTTP requests between microservices (getForObject(), exchange()).
 - Requires configuration as a Spring @Bean in the application class.
 - Uses **ParameterizedTypeReference** to correctly map complex return types (e.g., List<Long>).
 - Example:
 - Customer microservice:
 - Retrieves customer info.
 - Calls **Friend Service** for friend contact numbers.
 - Calls **Plan Service** for plan details.
 - Aggregates results into CustomerDTO and sends response back to the browser.
- **Scalability and Real-World Scope**
 - Any number of microservices can be developed and integrated.
 - Real projects may use **dozens of microservices** (e.g., 70+), maintained by multiple teams.

II. UI Application Integration

- **Purpose:** Provides a front-end for interacting with microservices (not part of Spring Boot services).
- **Key Dependencies:**
 - Web dependency.
 - Jasper + JSTL for JSP rendering.
 - No Spring Data JPA (UI doesn't connect to DB directly).

- **UI Application Structure:**
 - **Models (DTOs):** For communicating with microservices.
 - **Controller (Normal Controller):** Handles UI requests and calls microservices.
 - **JSP pages:** Login, profile, etc.
 - **Static assets:** Images and other UI resources.
- **Communication with Microservices:**
 - UI app also uses **RestTemplate** to call backend microservices.
 - Example: Login UI calls **Customer Service**, which in turn calls **Plan** and **Friend Services**.

III. Introduction to Spring Cloud Config Server

- **Problem in Microservices:**
 - Each microservice has its own application.properties.
 - Common properties (driver class, username, password, etc.) must be duplicated across all services.
 - Updating these across **dozens or hundreds** of services is inefficient.
- **Solution: Spring Cloud Config Server**
 - **Centralized Configuration:** Common properties stored in a single location.
 - **External Storage:** Configs managed in Git/Bitbucket repositories.
 - **Runtime Delivery:** Config Server delivers properties to microservices dynamically at runtime.
- **Advantages:**
 - **Simplified management:** Update config in one place only.
 - **Reduced overhead:** No need to redeploy each microservice for property changes.
 - **Consistency:** All services share the same version of common configs.

✓ Key Takeaway:

- Microservices communicate via **RestTemplate**, aggregate results in **customer service**, and expose them to a **UI app**.
- UI acts as a consumer layer, calling backend microservices instead of DBs directly.
- **Spring Cloud Config Server** solves configuration sprawl by centralizing shared properties in Git and serving them at runtime.

I. Cloud Config Server

- **Purpose & Role**

- Provides a **centralized repository** for common configuration properties.
- Reads from **Git repositories (e.g., GitLab)** and supplies properties at runtime.
- Removes duplication of configs across multiple microservices.
- Example: Database drivers, usernames, passwords managed centrally.

- **Key Quotes**

- *"Usage of cloud config server is in order to read the some common properties from G and hand overing to microservices at runtime."*
- *"Tomorrow 100 microservices are there... modify only in single place whatever you want. Then it is going to affected all the microservices."*

- **Implementation Steps (Server Side)**

1. Create a **Config Server project**.
2. Add **Cloud Version** (e.g., oxon.SR8) + spring-cloud-config-server dependency.
3. Annotate with @EnableCloudConfigServer.
4. Configure spring.cloud.config.server.git.uri → GitLab repository URL.

- **Implementation Steps (Client Side – Microservice)**

1. Add **Cloud Version** + spring-cloud-config-client dependency.
2. Create bootstrap.properties with:
 - spring.cloud.config.uri=http://localhost:6161/configserver.
3. Remove redundant properties from application.properties.
4. Microservices now fetch configs from Config Server at runtime.

- **Benefits**

- Centralized management of shared properties.
- Ensures **consistency** across all services.
- **Reduces duplication** of config files.
- **Dynamic updates** from GitLab → reflected in services automatically.

II. Load Balancing

- **Purpose**
 - Distributes requests across multiple service instances.
 - Prevents overload on a single microservice.
 - Improves **application performance and availability**.
- **Problem Example**
 - If customer service calls **friend service** repeatedly, a single instance may fail.
 - Solution: Add multiple instances of friend service (different ports).
- **Key Quotes**
 - *"In order to improve the application performance then we can go for load balancing."*
 - *"One more friend microservice you need to add... just change the port number and run it."*
- **Implementation Steps**
 1. Add spring-cloud-starter-netflix-ribbon dependency.
 2. Use @RibbonClient(name="custo") at controller or application level.
 3. Create @Bean RestTemplate and annotate with @LoadBalanced.
 4. Define server list in application.properties:
 - custo.ribbon.listOfServers=http://localhost:7373,http://localhost:8383.
 5. Call service via logical name (custo) instead of host:port.
- **Demonstration**
 - Requests alternate between instances (7373, 8383).
 - Confirms that **load balancing is active**.
 - Quote: *"7373 was providing service... next request 8383 provided service... definitely load balancing is working."*
- **Future Enhancement: Eureka Service Discovery**
 - Current approach = **manual server listing** in properties file.
 - Problems: Manual updates if ports/instances change.
 - Solution: **Eureka Service Discovery** dynamically registers and discovers microservice instances.

III. Conclusion and Scope

- Cloud Config Server and Load Balancing are **mini-projects** within a larger microservices project.
- Strong additions to a **resume portfolio**, particularly for telecom or distributed system domains.
- Upcoming topics:
 - **Eureka Service Discovery** (automated service registry).
 - **Zuul Server** (API Gateway).

✓ Key Takeaway:

- **Cloud Config Server** centralizes shared configs → Git as a single source of truth.
- **Load Balancing with Ribbon** distributes requests across multiple instances → boosts performance & fault tolerance.
- Together, they form the **foundation for scalable microservices**, to be extended with **Eureka + Zuul**.

1. EUREKA Service Discovery

1.1 Core Problem EUREKA Solves

- **Tight Coupling Issue:** Changes in host/port require updating all dependent microservices.
- **Decoupling Solution:** Register services with **logical names** (via Eureka) instead of hardcoding host/port.
- Quote: *"That means uh we can avoid the tightly coupled concepts in real time."*

1.2 EUREKA Server Implementation Steps

1. Create a **Spring Boot project**.
2. Add **Eureka server dependency** (spring-cloud-starter-netflix-eureka-server).
3. Enable Eureka server using @EnableEurekaServer.

1.3 Microservice Registration (Eureka Client)

1. Add **Eureka client dependency** (spring-cloud-starter-netflix-eureka-client).
2. Enable discovery with @EnableDiscoveryClient.
3. Set spring.application.name=<unique-name> in **application.properties** (used for registration).
 - Quote: *"Because of this property only, it is going to registered inside the Ureka server."*

1.4 EUREKA in Action

- **Registration:** Services automatically register with Eureka (dashboard at http://localhost:8761).
- **Discovery:** Services communicate using the registered **application name**, not host/port.
- **Flexibility:** Host/port changes do not affect other services as long as the service name remains constant.

2. ZUUL API Gateway

2.1 Core Problem ZUUL Solves

- **UI-Microservice Coupling:** UI apps directly tied to multiple microservices → complex and fragile.
- **Solution:** ZUUL provides a **single entry point** for external clients.
- Handles **routing, filtering, security, and centralized control**.
- Quote: *"In between we have to maintain the gateway server."*

2.2 ZUUL Server Implementation Steps

1. Create a **Spring Boot project**.

2. Add **Zuul dependency** (spring-cloud-starter-netflix-zuul).
3. Enable Zuul proxy with @EnableZuulProxy.
4. Enable discovery with @EnableDiscoveryClient (Zuul itself registers with Eureka).
5. Configure routes in application.properties:
 - Example:
 - zuul.routes.call.path=/call/**
 - zuul.routes.call.service-id=call-details

2.3 ZUUL in Action

- **Single Entry Point:** UI apps interact with Zuul only (e.g., http://localhost:6363).
- **Routing via Eureka:** Zuul forwards requests to services by looking up service IDs in Eureka.
- **Decoupling Advantage:** Changes in microservice endpoints are handled inside Zuul, UI apps remain unaffected.
- *Quote: "Zuul server is communicating to Eureka and Eureka is going to communicate internally to appropriate microservices."*

3. General Development Practices & Interview Advice

- **Spring Starter Project:** Always start with it for new microservices.
- **AI Tools (ChatGPT, Git Copilot):** Useful for code generation and problem-solving.
- **Core Java Knowledge:** Still critical for interviews and real-world development.
- **Outdated Technologies:** Servlets, JSP, JDBC mostly obsolete; Spring Boot REST APIs are the standard.
- **Deployment Tools:** Vary by project (Jenkins, Bamboo, GitLab, Bitbucket, Maven/Gradle/Ant). Learn them during KT sessions.
- **Environments:** Multiple environments (Dev, UAT, Prod) with separate servers are standard practice.

Key Takeaways

- **Eureka = Service Discovery:** Decouples microservices by registering with logical names.
- **Zuul = API Gateway:** Provides centralized routing, security, and single entry point for UI → microservices communication.
- Together, they make microservice architectures more **flexible, resilient, and manageable**.
- Career advice: Strong Java fundamentals + Spring Boot microservices + modern tools (Eureka, Zuul, Jenkins, Git) are essential.

1. Eureka Server for Service Discovery

Main Theme: Achieving *loosely coupled* microservices by centralizing service registration and discovery.

- **Problem (Tightly Coupled):**

- Microservices depend on fixed host/port of others.
- Any change breaks communication → tightly coupled.
- Quote: *"Tightly coupled... whenever we are doing the some changes in one microservice there is effect for the other microservices."*

- **Solution (Loosely Coupled):**

- Services register with **Eureka** using a logical name.
- Other services communicate by **name**, not host/port.
- Quote: *"Whenever we are changing the host and port there is no effect for the other microservices."*

- **Implementation Steps (Eureka Server):**

1. Create a Spring Boot project.
2. Add **eureka-server** dependency.
3. Annotate main class with **@EnableEurekaServer**.

- **Implementation Steps (Eureka Client):**

1. Add **eureka-client** dependency.
2. Annotate main class with **@EnableDiscoveryClient**.
3. Set **spring.application.name=<unique-name>** in **application.properties**.

- **Discovery Process:**

- Microservice registers automatically at startup.
- Others call it via registered name (e.g., plan-details) instead of http://localhost:port.
- Eureka dashboard (http://localhost:8761) shows registered services and status.

2. Zuul Server as API Gateway

Main Theme: Decoupling UI applications from microservices, centralizing routing through a single entry point.

- **Problem (Direct UI–Microservice Communication):**

- UI must know multiple microservice endpoints.
- Any backend change requires UI updates.
- Quote: *"If you are doing some changes in UI application, you need inform to microservice... This is very difficult."*
- **Solution (Zuul as Gateway):**
 - UI communicates only with **Zuul server**.
 - Zuul routes requests to backend microservices.
 - Backend changes do **not** impact the UI.
 - Quote: *"Jewel server... you no need to inform the UI application, it is the gateway."*
- **Integration with Eureka:**
 - Zuul registers itself with Eureka.
 - Uses Eureka for service lookup to route requests.
- **Request Flow:**
 - **Client → UI → Zuul → Eureka → Microservice**
- **Implementation Steps (Zuul Server):**
 1. Create Spring Boot project.
 2. Add **spring-cloud-starter-netflix-zuul** dependency.
 3. Enable proxy with `@EnableZuulProxy`.
 4. Register with Eureka using `@EnableDiscoveryClient`.
 5. Configure routes in **application.properties**:
 - Example:
 - `zuul.routes.call.path=/call/**`
 - `zuul.routes.call.serviceld=call-details`

3. OAuth2 for REST Endpoint Security

Main Theme: Securing REST APIs with token-based authorization.

- **Purpose:** Protect REST endpoints that are otherwise unprotected.
- Quote: *"In order to provide the security for the rest end points... then we can go for [OAuth2]."*
- **Core Components:**

1. **Client Application** (end user/resource owner).
 2. **Authorization Server** (authenticates, issues tokens).
 3. **Resource Server** (validates tokens, serves data).
- **Request Flow (Simplified):**
 1. **User Authentication:** Client prompts for username/password.
 2. **Authorization Code Grant:** User approves → authorization code returned.
 - Quote: *"If you click on the approve then we will get the authorization code."*
 3. **Token Exchange:** Client sends code + client ID/secret → Authorization Server.
 4. **Token Issuance:** Authorization Server issues:
 - **Access Token** (short-lived, for resource access).
 - **Refresh Token** (to renew access tokens).
 - Quote: *"Token is divided into two type. Refresh token and access token."*
 5. **Resource Access:** Client calls API with access token.
 6. **Token Validation:** Resource Server checks with Authorization Server.
 7. **Resource Delivery:** If valid, resource returned.
 - **Dependencies Required:**
 - spring-boot-starter-oauth2-client
 - spring-boot-starter-security
 - **Outcome:**
 - Direct endpoint access blocked.
 - Only requests with **valid tokens** are allowed.

Key Takeaways

- **Eureka → Service Discovery** → avoids tight coupling by registering services by name.
- **Zuul → API Gateway** → central entry point, decouples UI from microservices.
- **OAuth2 → Security** → token-based protection for REST APIs.
- Combined, they enable a **flexible, scalable, and secure microservices architecture**.

Part 1: OAuth2 Implementation for Secure REST Endpoints

Main Theme: OAuth2 is a third-party library essential for securing REST endpoints through an authorization flow that issues tokens after user authentication.

Key Concepts & Implementation Steps:

- **Necessity of OAuth2 for Security:**
 - REST endpoints previously accessible directly now require authentication.
 - Quote: “previously end points directly I can access but now we cannot access after implementing the oath 2.”
- **Core Components:**
 - **Spring Starter Project:** Create a project with spring-web and OAuth2 library.
 - **Rest Controller:** Define secure endpoints (e.g., /API/user/me) using @RestController.
 - **Model/DTO:** Create UserProfile model with user properties, getters, and setters.
 - **Authorization Server:**
 - Verifies user validity; returns an authorization code if valid.
 - Extends AuthorizationServerAdapter and overrides configure method.
 - Defines client details (client_id=Flyclient, scope=read, grant_type=authorization_code, refresh_token, password).
 - **Security Configuration:**
 - In-memory user credentials (username, password=123456).
 - Handles form login automatically.
 - **Resource Server:** Enabled via @EnableResourceServer; no additional coding needed.
 - **application.properties:** Stores app-specific configurations.
- **Authorization Flow Steps:**
 1. **Direct Access Attempt:** Accessing protected endpoint returns “full authentication is required.”
 2. **Generate Authorization Code:** Redirects to login page via URL with client_id and scope.
 3. **User Authentication:** Enter credentials on login page.
 4. **Approve/Deny Request:** User approves; leads to authorization code issuance.
 5. **Receive Authorization Code:** Code returned in callback URL.

6. **Generate Access Token:** Use POST request (e.g., Postman) with authorization code; receives access and refresh tokens.

- Access token validity: 100 ms
- Refresh token validity: 300 ms

7. **Access Protected Resource:** Use token to access endpoints.

8. **Token Expiration/Invalidity:** Expired or wrong token returns “invalid access token.”

Part 2: Introduction to Kafka for Asynchronous Message Handling

Main Theme: Kafka is a robust, asynchronous message system for handling large volumes of data efficiently, surpassing older systems like RabbitMQ and JMS.

Key Concepts & Advantages:

- **Message-Oriented System:** Sends and receives messages asynchronously.
- **Asynchronous Communication:** Supports two-way message handling in near real-time.
- **High Volume Handling:** Can process large amounts of data in fractions of a second.
- **High Reliability:** “99.999% no failure” in message sending and receiving.
- **Pre-Implemented Software:** Users leverage Kafka like Tomcat/WebLogic; no need to develop underlying server logic.

Kafka Architecture & Components:

- **Kafka Cluster:** The overall system managing brokers and topics.
- **Brokers (Servers):** Multiple brokers (Broker 0, 1, 2) provide redundancy.
- **Zookeeper Server:**
 - Manages and monitors cluster.
 - Assigns tasks in case of broker failures.
- **Topics:** Logical groupings of messages (e.g., GPS_track_Python_GPS).
- **Partitions:** Each topic divided into partitions (P0, P1, P2) for message storage.
- **Redundancy:**
 - Intra-broker: Partition failure handled by another partition.
 - Inter-broker: Broker failure managed by Zookeeper.

Practical Example:

- Kafka can send bulk alerts to over 10 million people efficiently, which would be challenging with RabbitMQ or JMS.

High-Level Implementation Steps:

1. Download Kafka software.
2. Start Zookeeper server.
3. Start Kafka brokers.
4. Create partitions within topics (detailed implementation in subsequent sessions).

Main Themes

- **Step-by-Step Kafka Setup:** Sequential process for setting up and configuring Kafka for applications.
- **Core Kafka Components:** Roles of Zookeeper, Kafka Server (Broker), Topics, and Partitions.
- **Producer-Consumer Model:** Asynchronous sending and receiving of messages.
- **Spring Boot Integration:** Kafka integrated into a Spring Boot application using dependencies and classes.
- **Scalability & Resilience (Implicit):** Concepts like replication factors, leader/follower partitions, and multiple Kafka servers address fault tolerance and scalability.

Key Concepts & Implementation Steps**1. Kafka Software Download and Initial Setup**

- Prerequisite: Download Kafka software before implementation.
- **Server Startup Order:**
 1. Start **Zookeeper server** first (manages Kafka cluster).
 2. Start **Kafka server (Broker)** afterwards.
- Zookeeper crucial for cluster management.

2. Starting Kafka Components via Command Line

- **Zookeeper Start Command:** `./bin/windows/zookeeper-server-start.bat config/zookeeper.properties`
- **Zookeeper Default Port:** 2181
- **Kafka Broker Start Command:** `./bin/windows/kafka-server-start.bat config/server.properties`
- **Kafka Default Port:** 9092
- **Broker ID:** Unique for each instance, starting from 0
- **Shutdown Issues:** Restart Zookeeper and Kafka if server fails to recognize cluster

3. Creating Kafka Topics and Partitions

- **Topic Creation Command:**
`./bin/windows/kafka-topics.bat --create --topic demo --partitions 3 --replication-factor 1 --zookeeper localhost:2181`
- **Topic Name Example:** demo
- **Partitions:** Enable parallelism; example creates 3 partitions

- **Leader and Followers:** One leader per partition; others are followers. Leader handles messages; if it fails, a follower takes over
- **Replication Factor:** Number of servers replicating a topic's data
- **Describe Topic Command:**
./bin/windows/kafka-topics.bat --describe --topic demo --zookeeper localhost:2181

4. Implementing Kafka in Spring Boot

- **Application Structure:** One controller, two services: Producer & Receiver (Consumer)
- **Dependencies:** spring-boot-starter-web and Kafka dependency
- **KafkaTemplate:** Predefined class for sending messages in producer & consumer
- **Producer Service:** Uses KafkaTemplate.send() to send messages to topic demo
- **Receiver Service:** Uses @KafkaListener to listen to messages on specific topic
- **Group ID:** Organizes consumers; multiple instances can share the same group ID
- **application.properties Configuration:**
 - Application port & context path
 - Kafka server bootstrap (spring.kafka.bootstrap-servers)
 - Consumer group ID (spring.kafka.consumer.group-id)
 - Consumer behavior (spring.kafka.consumer.auto-offset-reset=latest)

5. Application Execution & Message Flow

- **Startup Sequence:** Start Zookeeper → Kafka → Spring Boot app
- **Restarting Issues:** Restart Kafka & Zookeeper if application fails to recognize Kafka
- **Message Sending:** HTTP POST request triggers producer to send message to demo topic
- **Message Receiving:** HTTP GET request triggers consumer to receive messages
- **Asynchronous Nature:** Producer does not wait for consumer confirmation
- **Scalability Across Services:** Same topic & group ID can be reused across multiple controllers/services

6. Career Implications

- Knowledge of Kafka with Spring Boot & Microservices is valuable for IT professionals
- Demonstrates practical expertise suitable for 5+ years of experience
- Potential for competitive industry salaries (e.g., ~20 lakhs per annum in local currency context)

Main Themes

- **Transition from Spring JDBC → ORM/JPA**
 - JDBC required manual SQL + schema management.
 - ORM (Hibernate, JPA, Spring Data JPA) automates persistence, schema, and query handling.
- **Performance Boost**
 - Data transferred as **objects** instead of text/field-by-field cycles.
 - Reduces round trips and increases efficiency.
- **Simplified Development**
 - SQL & table creation handled by framework.
 - Developers focus on business logic, not low-level DB tasks.
- **Essential Components**
 - POJO/Entity classes.
 - Mapping (XML or Annotations).
 - Configuration files/properties.

1. Limitations of Spring JDBC

- Developer must:
 - Write SQL queries for each DB (Oracle, MySQL, etc.).
 - Manually design database structure.
- Data transfer is **text-based & repetitive** (slow with many columns).
- No clear confirmation of data persistence.
- ORM solves these challenges.

2. Benefits of Spring with ORM

- **Performance:** Data transferred as **objects** (entities).
- **Reduced effort:** Framework handles queries + table creation.
- **Entity Representation:** DB record ↔ Java object (@Entity).
- **Flexible Mapping:** Supports both XML and Annotations (modern approach favors annotations).

3. Components of ORM Applications

- **Entity Class (POJO):**
 - Marked with @Entity.
 - Primary key: @Id.
 - Columns: @Column(name="EMP_NAME", length=10).
- **Mapping:**
 - XML <hibernate-mapping> or annotations in entity class.
 - Defines how object fields map to DB columns.
- **Configuration File (annotations preferred):**
 - Uses @Configuration, @PropertySource, @ComponentScan.
 - Environment object reads properties (env.getProperty("db.driver")).

4. Hibernate/ORM Properties

- **hibernate.hbm2ddl.auto:**
 - create → Creates tables every run (drops old data).
 - update → Updates schema if needed (recommended for development).
 - create-drop → Creates + drops tables each run (dangerous in production).
- **hibernate.show_sql:** Shows generated SQL → confirms DB operations.

5. Key Configuration Classes

- **DataSource:** Connection setup (driver, URL, username, password).
 - Examples: DriverManagerDataSource, BasicDataSource.
- **LocalSessionFactoryBean:** Bridges Spring + Hibernate. Requires:
 - DataSource.
 - Hibernate properties.
 - Entity classes or mapping files.
- **HibernateTransactionManager:** Handles transactions with SessionFactory.

6. Performance & Requirements

- Focus on **performance + meeting client needs**.
 - Without performance, apps won't be adopted.
-

7. Object Properties

- Every object has:
 - **Identity** (name, not stored in DB).
 - **Behavior** (methods like save/delete, not stored in DB).
 - **Values** (stored in DB).
-

8. Design Patterns (Planned Coverage)

- MVC, DAO, DTO, Singleton → important in Spring ORM development.
-

9. Object Creation

- **Entities must be created with new** → ensures unique objects.
 - **Don't Autowire entity classes** → would make them singletons, invalid for representing DB records.
-

✓ This document captures the **shift from low-level JDBC to high-level ORM** in Spring, stressing performance, reduced effort, and proper design practices with Hibernate/Spring Data JPA.

1. Spring Request Flow (Step-by-Step)

- **User Input → Browser Request:** e.g., login.jsp with username/password.
- **Web Container:** First receives the request, loads **web.xml**.
- **Dispatcher Servlet (Front Controller):** Configured in web.xml, central entry point.
- **Configuration Loading:** Dispatcher Servlet loads its own config file (e.g., disp-servlet.xml) containing beans.
- **Handler Mapping:** Forwards request to **SimpleUrlHandlerMapping**.
- **Controller Identification:** Finds appropriate sub-controller (e.g., LoginController).
- **Handler Execution:** Controller method runs business logic.
 - Small logic inside controller.
 - Complex logic via **DAO/Service layer**.
- **Model and View Return:** Controller returns a **ModelAndView** (view name + data).
- **View Resolver:** Dispatcher Servlet passes logical view name to **InternalResourceViewResolver**.
 - Appends prefix & suffix (e.g., /WEB-INF/views/ + success + .jsp).
- **Final Response:** Dispatcher Servlet returns physical view (e.g., success.jsp) to browser.

Traditional Spring requires manual configuration: DispatcherServlet, HandlerMapping, Controllers, ViewResolver.

2. Spring Boot – Key Advantages vs. Spring

- **Built on Spring Framework** but simplifies setup.
- **Zero/Less Configuration:** Auto-configures DispatcherServlet, controllers, handler mappings, view resolvers.
- **Embedded Servers:** Tomcat/Jetty/Undertow built-in → no external deployment needed.
- **Convention over Configuration:** Sensible defaults, minimal XML/Java config.
- **Starter Templates:** Simplified project setup.
- **Microservices Friendly:** Preferred for modern architectures.
- **Industry Standard:** Widely used in IT.

When to use:

- **New Projects** → Spring Boot.

- **Legacy Projects** → Traditional Spring (may still rely on JDBC/ORM, XML configs).

3. Controller Design Approaches

- **Old (Spring):**
 - Implement Controller interface.
 - Extend AbstractCommandController.
 - Single handleRequest with multiple parameters.
- **Modern (Spring & Boot):**
 - **@Controller annotation** (preferred).
 - Mapping annotations:
 - @RequestMapping(method=GET/POST)
 - @GetMapping, @PostMapping, @PutMapping, @DeleteMapping.
 - Multiple handler methods inside one controller → cleaner, more organized.

4. Handling Data and Views

- **Request Parameters:**
 - Auto-binding to POJO (e.g., LoginCommand with username/password).
 - Or via request.getParameter("t1").
- **Passing Data:**
 - Add to **ModelAndView** (addObject("username", uName)).
 - Access in JSP via \${username} or request.getAttribute("key").
- **View Layer:** JSP shown for demo, but real-world often uses modern front-ends.

5. Database Communication (Spring JDBC Example)

- DAO/Service layer encapsulates DB operations.
- Uses **JdbcTemplate** for simplified queries.
- Dependencies chain:
 - HandlerMapping → Controller → Service (DAO) → JdbcTemplate → DataSource.
- Traditional Spring required configuring all five components manually.

6. Spring Security Basics

- Used for **authentication & authorization**.
- Configuration requires:
 - Security config file.
 - Filter chain.
- Filter checks if user exists; valid credentials → session ID generated.
- Supports **in-memory authentication** and other mechanisms.
- Covered more deeply in Spring Boot later.

7. Core Spring Concepts (Foundation for Boot)

- **Core Module:** Dependency Injection (DI).
- DI types:
 - Setter Injection (most common).
 - Interface Injection.
 - Constructor Injection.
 - Lookup Injection.
- Knowledge of core Spring is **mandatory** since Boot apps internally run as Spring apps.

Conclusion

- **Spring Boot** drastically reduces boilerplate by automating configuration and embedding servers.
- **Spring Framework** concepts are still essential for understanding Boot internals.
- Boot is the go-to for **new projects and microservices**, while Spring Framework knowledge is needed for **legacy maintenance and deeper debugging**.

1. What is Spring Boot and How Does It Differ from Spring?

- Extension of Spring Framework, designed to simplify Java development.
- **Key Differences:**
 - Built on Spring Framework.
 - Simplifies Java development with **auto-configuration**, **embedded servers**, and **starter dependencies**.
 - Reduces XML configuration and boilerplate code.

2. Main Features of Spring Boot

- **Auto-Configuration (Zero Configuration):** No manual configuration required.
- **Starter Dependencies:** Pre-configured JAR collections for common features.
- **Embedded Servers:** Includes embedded Tomcat or Jetty; no separate installation needed.
- **Actuators:** Monitoring tools primarily for production teams.
- **Easy Microservice Development:** Facilitates creation of microservices.

3. @SpringBootApplication

- Predefined annotation combining three annotations:
 1. **@Configuration:** Marks class as a source of bean definitions.
 2. **@EnableAutoConfiguration:** Enables Spring Boot auto-configuration.
 3. **@ComponentScan:** Scans current package and sub-packages for Spring components.

4. @ComponentScan

- Scans base package and sub-packages for Spring components.
- Provides information to the container for object creation and dependency injection.
- Example: Base package com.dura scans com.dura.beans, com.dura.service, com.dura.service.impl, etc.

5. Starter Dependencies

- Simplify dependency management in Spring Boot.
- **Collection of JARs:** Pre-packaged compatible versions of dependencies.
- **Examples:**
 - spring-boot-starter-web: REST, embedded Tomcat.
 - spring-boot-starter-data-jpa: JPA, Hibernate.

6. Auto-Configuration

- Reduces boilerplate configuration.
- Allows developers to focus on Java beans implementation.
- Spring Boot automatically handles configuration behind the scenes.

7. Configuring Properties

- **application.properties:** Key-value pair configurations.
- **application.yml:** Hierarchical, more readable for complex configs.
- **@Value annotation:** Injects property values into fields (@Value("\${property.name}")).
- **@ConfigurationProperties annotation:** Binds external configuration properties to Java objects.

8. Difference Between application.properties and application.yml

- **application.properties:** Simple key=value pairs.
 - Example:
 - server.port = 8985
 - spring.datasource.driver-class-name = com.Oracle.driver.OracleDriver
- **application.yml:** Hierarchical, eliminates repetitive data, easier to read.
 - Example:
 - datasource:
 - url: jdbc:oracle:thin:@localhost:1521/ServiceIdx

9. Changing Default Port

- Default port is 8080.

- Change via application.properties:
 - server.port = 9095
-

10. Injection Types in Spring Boot

- **Constructor Injection:** Dependencies via constructor.
- **Setter Injection:** Dependencies via setter methods (most preferred).
- **Field Injection:** Dependencies directly into fields with @Autowired (not preferred).

I. Spring Boot Application Troubleshooting and Best Practices**A. Common Issues and Resolutions**

- **Hands-On Practice:** Students must implement solutions themselves; practical experience is critical.
 - Quote: "You have to implement right? If you are not implemented mean that is your problem."
- **Port Number Conflicts:** Adjust application port if conflicts arise.
 - Quote: "Can I change it? Your choice. You can change not any."
- **Configuration File Adjustments:** Comment out problematic lines in WebSecurityConfiguration to resolve security-related issues.
- **Database Schema Mismatches:**
 - Error arises if table columns don't match application data.
 - Example: EMP table expected 5 values but has 4 columns → "Field envs does not have default value."
- **Resolution Strategies:**
 - Drop problematic columns: `ALTER TABLE table_name DROP COLUMN column_name.`
 - Drop and recreate the entire table if issues persist.
- **Data Source Configuration:** Temporarily comment out data source configuration to recreate the database.
- **Server Management:** Restart server after code changes to ensure proper functionality.
 - Quote: "Restart the server and try to insert... Server also not started. You have to restart the server."
- **Post-Resolution Verification:** Confirm that functionalities like list, edit, and delete operations are working correctly.

B. Instructor Emphases and Warnings

- **Self-Reliance:** Students must develop independence; industry support is limited.
 - Quote: "Nobody will help in the industry... nobody will help frankly I'm saying."
- **Time Constraints:** Industry expects quick deliverables; one week may be given to complete work that took 30+ days in learning.
 - Quote: "In the industry just they are going to give... only one week time..."

II. Monolithic vs. Microservices Architecture**A. Monolithic Application Characteristics and Drawbacks**

- **Definition:** Single application providing multiple services.
- **Single Point of Failure:** Failure in one functionality can halt the entire application.
 - Example: SBI app with withdraw, check balance, transfer balance – failure in one function stops all services.
- **Business Impact:** Downtime leads to profit loss.
 - Quote: “If withdraw is not working... client is going to losing lot of profit.”

B. Microservices Architecture as a Solution

- **Definition:** Monolithic application divided into multiple smaller applications (microservices).
- **Independent Services:** Each functionality becomes a microservice.
- **Improved Resilience:** Failure in one microservice does not affect others.
 - Example: Transfer balance fails, withdraw/check balance still operational.
- **Development Requirements:** Foundational knowledge from previous sessions is essential.
 - Quote: “Whatever we are learning so far... everything is required that API is required in order to develop this microservices.”
- **Communication Between Services:** Microservices must communicate efficiently; inter-service communication can be complex.
 - Quote: “Functionality one is going to communicate into functionality... Functionality 2 is going to communicate into functionality four.”

III. Administrative Notes and Future Sessions

- **Session Access:** Session 11 will be re-uploaded on request via email.
- **Next Session:** Scheduled for Thursday; no session tomorrow.

1. Eureka Service Discovery

Core Concept:

- Addresses challenges of tightly coupled microservices; services can discover each other using a logical name instead of host/port.

Key Advantages:

- Decoupling:** Services don't depend on hardcoded host/port.
 - Quote: "Instead of using that host and port directly we can use some name..."
- Flexibility:** Host/port changes don't affect other services.
- Real-time Applications:** Supports dynamic environments with scaling or network changes.

Implementation Steps – Eureka Server:

- Create a Spring Boot project.
- Add dependency: spring-cloud-starter-netflix-eureka-server.
- Enable server: Add @EnableEurekaServer annotation.

Implementation Steps – Eureka Clients (Microservices):

- Add dependency: spring-cloud-starter-netflix-eureka-client.
- Enable discovery: Add @EnableDiscoveryClient annotation.
- Register application name in application.properties:
 - Example: spring.application.name=all-details
 - Quote: "Because of this property only, it is going to registered inside the Eureka server."

Communication Flow:

- Microservices call each other using registered names (e.g., plan-details) instead of host/port.
- Eureka facilitates name-to-instance resolution.

2. Zuul Gateway Server

Core Concept:

- Acts as an API Gateway between UI and microservices, centralizing routing and decoupling the UI.

Key Advantages:

- Decoupling UI from Microservices:** UI only communicates with Zuul, not individual services.

SPRING BOOT & MICRO SERVICES

- Quote: “If you are doing some changes to this microservices informing to that UI application is not a good approach...”
- **Centralized Routing:** Routes all incoming requests to the appropriate microservice.
- **Simplified UI Development:** UI only needs Zuul URL and configured paths.
 - Quote: “Always I'm using Juul server port only...”

Implementation Steps – Zuul Server:

1. Create a Spring Boot project.
2. Add dependency: spring-cloud-starter-netflix-zuul.
3. Enable Zuul proxy: Add @EnableZuulProxy.
4. Enable Eureka discovery: Add @EnableDiscoveryClient.
5. Configure routes in application.properties:
 - Example:
 - `zuul.routes.call.path=/call/**`
 - `zuul.routes.call.service-id=call-details`

Communication Flow:

- UI sends requests to Zuul port (e.g., 6363), which routes to the correct microservice via Eureka.

3. General Development Practices and Tools

- **Spring Starter Projects:** All microservices, Eureka, and Zuul start as Spring Starter Projects.
- **Core Java:** Strong fundamentals essential for Spring Boot and microservices roles.
- **AI Tools:** ChatGPT, Git Copilot assist coding but understanding core concepts is essential.
- **Outdated Technologies:** JSP, Servlets, JDBC rarely used; Spring Boot REST APIs are the standard.
- **Deployment and CI/CD:** Tools like Jenkins, Bamboo, GitLab/Bitbucket, multiple environments (Dev, UAT, Prod).
- **DevOps Team Responsibility:** Typically handles production deployment; developers monitor and fix issues.

4. Key Takeaways

- Eureka enables dynamic service discovery and prevents tight coupling.
- Zuul simplifies UI interaction with microservices and centralizes routing.

SPRING BOOT & MICRO SERVICES

- Strong Core Java knowledge is crucial.
- AI tools can improve efficiency but don't replace foundational understanding.
- Deployment processes are organization-specific and usually taught internally.

1. Creating REST APIs in Spring Boot

- **Core Annotation:** @RestController is used to create REST APIs.
 - Quote: "In order to create the rest API in Spring Boot, we have to use at the rate of RestController annotation."
- **Structure:** Class annotated with @RestController; methods use mapping annotations like @GetMapping, @PostMapping, etc.
- **Example:**
- @RestController
- public class HelloController {
- @GetMapping("/hello")
- public String sayHello() {
- return "hello Spring and Microser.";
- }
- }
- **Return Type:** Returns data (JSON/XML); @Controller is used for views.

2. Database Connectivity in Spring Boot

- **Dependencies:** spring-boot-starter-data-jpa required for database access.
 - Quote: "First, in order to connect with the database, we have to use following starter dependency: Spring Boot Starter Data JPA."
- **Configuration:** Defined in application.properties.
 - spring.datasource.url → JDBC URL (e.g., jdbc:mysql://localhost:3306/testdb)
 - spring.datasource.username → DB username (e.g., root)
 - spring.datasource.password → DB password (e.g., admin)
 - spring.jpa.hibernate.ddl-auto=update → Updates schema without dropping existing data.

3. Core Spring Boot Annotations

- **@Component:** Marks a class as a Spring bean.
 - Quote: "@Component is used for in order to make one ordinary class, a Spring bean."

- **@Service:** Marks a class as a business component.
- **@Repository:** Marks a class as a DAO/persistence class.
- **@Controller:** Marks a class as a web controller (returns views).
- **@RestController:** Combines @Controller + @ResponseBody for REST APIs (returns JSON/XML).

4. Spring Boot Security

- **Dependency:** spring-boot-starter-security.
 - Quote: "In order to handle security... we have to add Spring-Boot-Starter-Security."
- **Types of Security:**
 - In-Memory Authentication → Small applications.
 - JDBC Authentication → Large applications with DB-stored credentials.

5. Monitoring Spring Boot Applications

- **Tools/Concepts:**
 - **Spring Boot Actuators:** Health checks, metrics, info, environment properties.
 - **Micrometer:** Application metrics facade for multiple monitoring systems.
 - **Logback:** Logging framework commonly used.

6. Spring Boot Support for Microservices

- **Embedded Servers:** Simplifies deployment (Tomcat, Undertow).
- **RESTful APIs:** Facilitates inter-service communication.
- **Spring Cloud Tools:**
 - Service Discovery (Eureka, Consul)
 - Config Server for centralized configuration
 - API Gateway (Spring Cloud Gateway)
 - Actuators for monitoring individual microservices

7. Spring Boot Auto-Configuration

- **Mechanism:** Automatically configures beans based on classpath dependencies.

- Quote: “Spring Boot automatically configures beans based on dependencies present in the classpath.”
 - **Example:** Adding spring-boot-starter-web configures DispatcherServlet, embedded Tomcat, etc., without explicit XML/Java config.
-

8. Creating Entities

- **Annotation:** @Entity marks a class as a database entity.
 - Quote: “In order to make one class as an entity, we have to use @Entity annotation.”
- **Structure:** Fields represent DB columns; include an identifier field (id, accountNumber).

- **Example:**

- @Entity
- public class Customer {
- private int id;
- private String accountNumber;
- private String customerName;
- // Getters and setters
- }

I. What are Microservices?

- **Definition:** Applications built as "a collection of loosely coupled independently deployable services."
- **Contrast with Monolithic Applications:**
 - Monolithic: Single application with all functionalities tightly packaged and deployed together.
 - Microservice: Small, focused application, often with its own database.
- **Key Characteristics:**
 - Loosely Coupled: Minimizes interdependencies between services.
 - Independently Deployable: Each service can be deployed, updated, and scaled independently.
 - Single Business Capability: Each microservice focuses on one function.
 - Separate Database: Each service may have its own DB for data isolation.

II. Benefits of Microservices

- **Scalability:** Services can be scaled independently according to demand.
- **Independent Deployment:** Faster and simpler deployments; no need to redeploy entire application.
- **Fault Isolation:** Failure in one service less likely to affect the entire system.
- **Faster Development/Deployment:** Enables agile and rapid development cycles.

III. Implementing Microservices in Java

- **Core Frameworks:**
 - **Spring Boot:** Creates stand-alone, production-grade applications.
 - **Spring Cloud:** Provides tools for distributed systems (service discovery, config, circuit breakers).
 - **Micronaut:** Alternative framework for microservices.

IV. Microservices Communication

- **Communication Types:**
 - **Synchronous:** REST/HTTP request-response model.

SPRING BOOT & MICRO SERVICES

- **Asynchronous:** Services communicate without waiting for immediate response; includes:
 - **Kafka:** Preferred for event-driven messaging.
 - **RabbitMQ:** Largely obsolete in current practice.
 - **JMS:** Standard message-oriented middleware.
 - **Event-Driven Architecture:** Services publish/subscribe to events for loose coupling and scalability.
 - **Data Sharing:** Each microservice has its own DB; data exchange via events.
-

V. Key Microservices Concepts and Patterns

- **Service Discovery:** Mechanism for dynamically locating services (e.g., Eureka, Zookeeper).
 - **Spring Cloud Config Server:** Centralized configuration management for all microservices.
 - **Circuit Breaker:** Prevents cascading failures; stops requests to failing services.
 - **Distributed Transactions:** Handled using **Saga pattern** for operations spanning multiple services.
 - **API Gateway:** Central entry point for clients; routes requests and can secure inter-service communication.
-

VI. Testing and Monitoring Microservices

- **Testing Frameworks:**
 - JUnit 5
 - Spring Boot Test
 - **Monitoring:**
 - Spring Boot Actuator for health checks, metrics, and information endpoints.
-

VII. Security in Microservices

- **Securing Inter-Service Communication:**
 - **Mutual TLS (mTLS):** Two-way authentication and encryption.
 - **JWT Tokens:** Secure authentication and information exchange.
 - **API Gateway:** Handles authentication/authorization at the entry point of services.
-